Binary Arithmetic

BINARY ADDITION

- ✓ **Two 1-bit values:**

A	B	A + B
0	0	0
0	1	1
1	0	1
1	1	10

"two"

BINARY ADDITION

- ✓ **Two n-bit values:**
  - Add individual bits
  - Propagate carries
- ✓ **Example:**

```
  1  1  1  1  1  1  <-------- carries
     1  1  1  1  0  1          6  1
+    1  0  1  1  1       +     2  3
-----------------------    ----------
  1  0  1  0  1  0  0          8  4
```

BINARY ADDITION

- ✓ **Two n-bit values:**
  - Add individual bits
  - Propagate carries
- ✓ **Example:**

$$\begin{array}{r} {\scriptstyle 1}\quad\quad{\scriptstyle 1} \\ 10101 \\ +\ 11001 \\ \hline 101110 \end{array} \qquad \begin{array}{r} 21 \\ +\ 25 \\ \hline 46 \end{array}$$

BINARY SUBTRACTION

- ✓ **Two 1-bit values:**

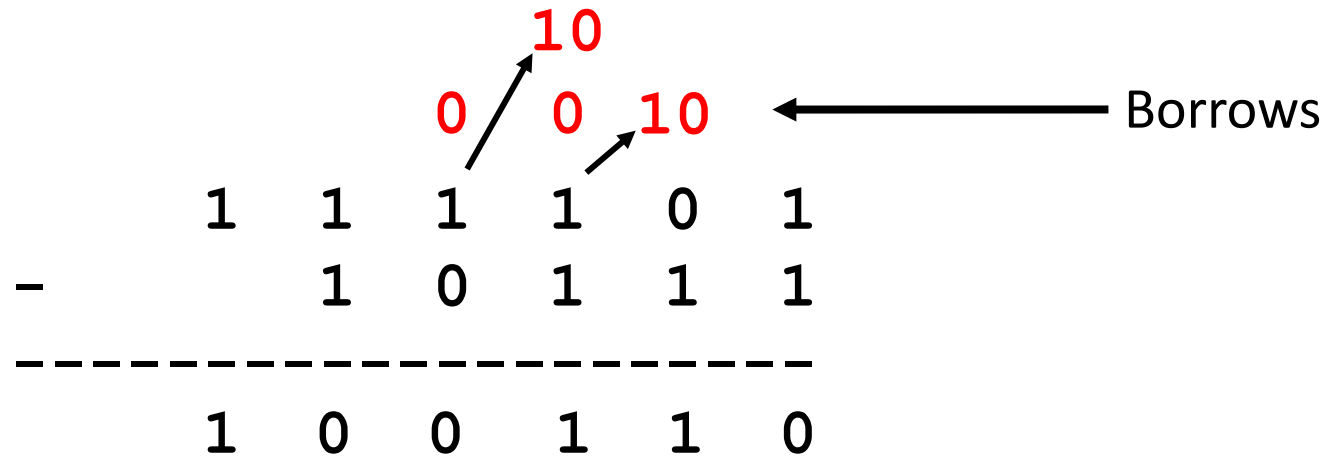
A	B	A + B
0	0	0
0	1	1
1	0	1
1	1	0

With Borrow 1

BINARY SUBTRACTION

- ✓ **Two n-bit values:**
  - Subtract individual bits with borrow if necessary
- ✓ **Example:**

10- decimal value 2




```
      6  1
-     2  3
----------
      3  8
```

BINARY MULTIPLICATION

- ✓ **Two 1-bit values:**

A	B	A × B
0	0	0
0	1	0
1	0	0
1	1	1

BINARY MULTIPLICATION

✓ **Example:**

```
                1   0   1   1   1
        x                   1   0
        1   0
        ---------------------------
                0   0   0   0   0
            1   0   1   1   1
        0   0   0   0   0
    1   0   1   1   1
    ---------------------------
    1   1   1   0   0   1   1   0
```

```
          2   3
      x   1   0
      ----------
        2   3   0
```

ASSIGNMENT QUESTIONS

1. Add: $(1000111)_2 + (1110)_2$
2. Subtract: $(1110000)_2 - (1111)_2$
3. Multiply: $(101)_2 \times (10101)_2$

COMPLEMENT OF NUMBERS

- ✓ In general, **complements** are used to **simplify the subtraction operation and for logical manipulation**.
- ✓ There two types of complement method in any base-b number system.
  - **b's Complement (Radix Complement)**
  - **(b-1)'s Complement (Diminished Radix Complement)**
- ✓ In base-2 number system
  - 2's complement
  - 1's complement
- ✓ In base-10 number system
  - 10's complement
  - 9's complement

DIMINISHED RADIX [(b-1)'s] COMPLEMENT

- ✓ If a number ***N*** having ***n*** digits with base ***b***, then the (b-1)'s complement is given by

$$(b^n - 1) - N$$

- ✓ For **binary number**, it is **1's complement** and can be represented as

$$(2^n - 1) - N$$

- ✓ For Example: 1's complement of $(10101)_2$

$$(2^5 - 1) - (10101)_2$$

$$\Rightarrow (32 - 1) - (10101)_2$$

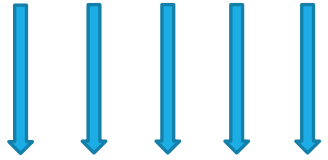
$$\Rightarrow (31) - (10101)_2 \qquad \Rightarrow (11111)_2 - (10101)_2 = (01010)_2$$

DIMINISHED RADIX [(b-1)'s] COMPLEMENT

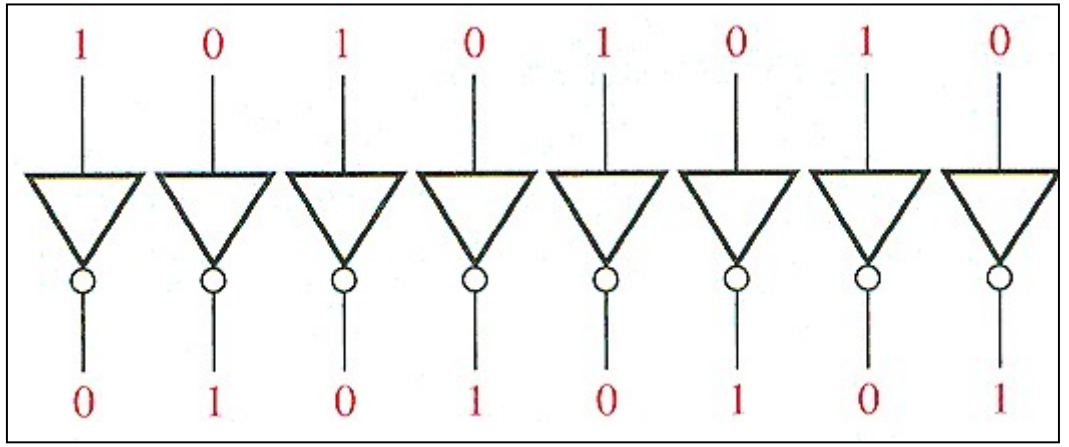
Note:

- ✓ **The 1's complement of a binary number can be obtained by changing each 0's to 1's and each 1's to 0's.**
- ✓ For example: 1's complement of $(11101)_2$

1 1 1 0 1



0 0 0 1 0



1's complement of $(11101)_2 = (00010)_2$

DIMINISHED RADIX [(b-1)'s] COMPLEMENT

- ✓ For **decimal number**, it is **9's complement** and can be represented as

$$(10^n - 1) - N$$

- ✓ For Example: 9's complement of $(54230)_{10}$

$$(10^5 - 1) - (54230)_{10}$$

$$\Rightarrow (100000 - 1) - (54230)$$

$$\Rightarrow (99999) - (54230)$$

$$\Rightarrow 45769$$

DIMINISHED RADIX [(b-1)'s] COMPLEMENT

Note:

- ✓ **The 9's complement of a decimal number can be obtained by subtracting each digit by 9.**
- ✓ For example: 9's complement of $(12345)_{10}$

$$\begin{array}{r} 9\ 9\ 9\ 9\ 9 \\ -\ 1\ 2\ 3\ 4\ 5 \\ \hline 8\ 7\ 6\ 5\ 4 \end{array}$$

9's complement of $(12345)_{10}$ = $(87654)_{10}$

RADIX (b's) COMPLEMENT

- ✓ If a number ***N*** having ***n*** digits with base ***b***, then the b's complement is given by

$$b^n - N = [(b^n - 1) - N] + 1 = \text{(b-1)'s Complement + 1}$$

- ✓ For **binary number**, it is **2's complement** and can be represented as

$$(2^n - N)$$

- ✓ For Example: 2's complement of $(10111)_2$

$$2^5 - (10111)_2$$

$$\Rightarrow 32 - (10111)_2$$

$$\Rightarrow (100000)_2 - (10111)_2 \quad \Rightarrow (01001)_2$$

1's complement of $(10111)_2$ = $(01000)_2$

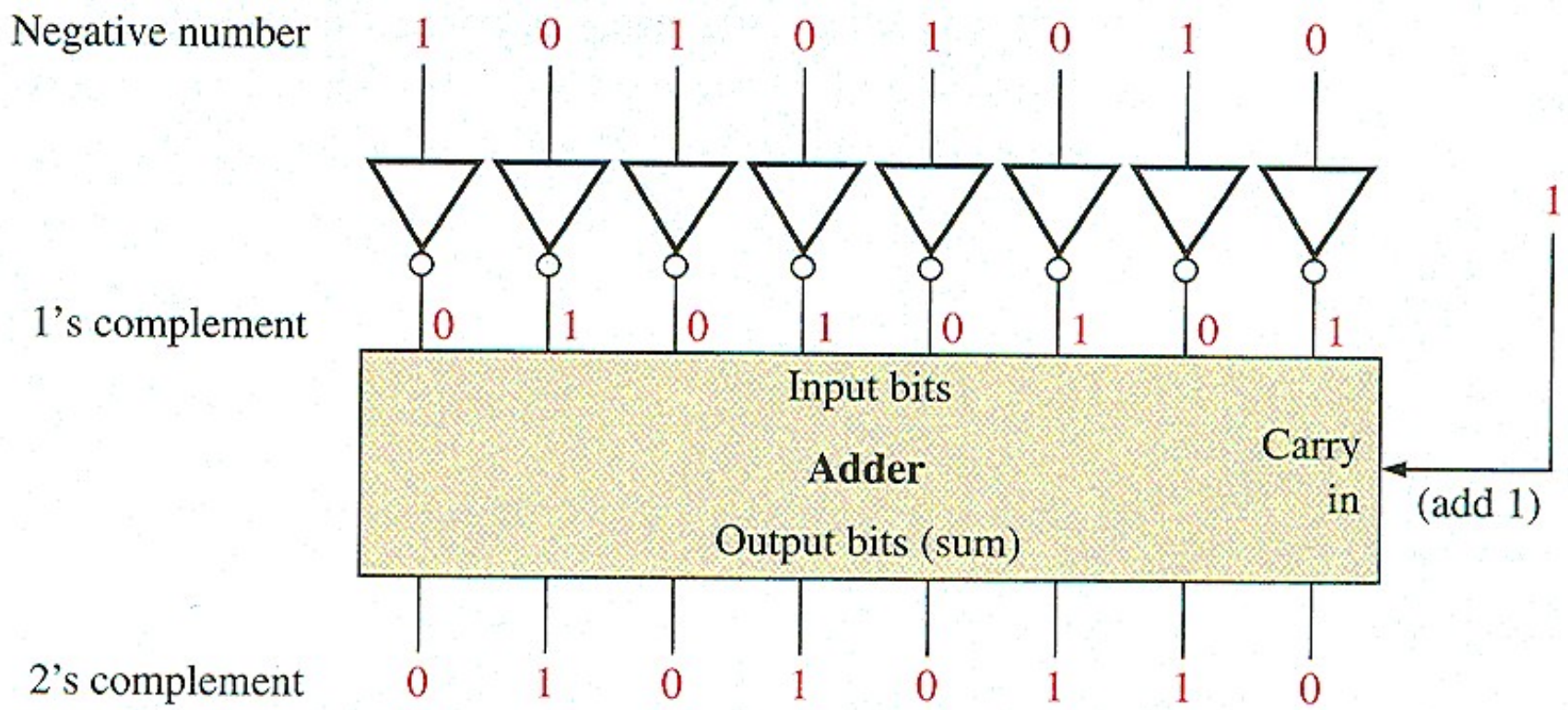
2's complement of $(10111)_2$

= 1's complement of $(10111)_2$ + 1

= $(01000)_2$ + 1 = $(01001)_2$

RADIX (b's) COMPLEMENT

- ✓ For **binary number**, it is **2's complement** and can be represented as

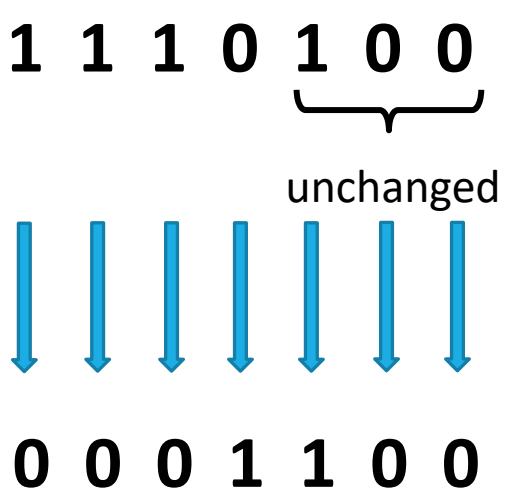


RADIX (b's) COMPLEMENT

Note:

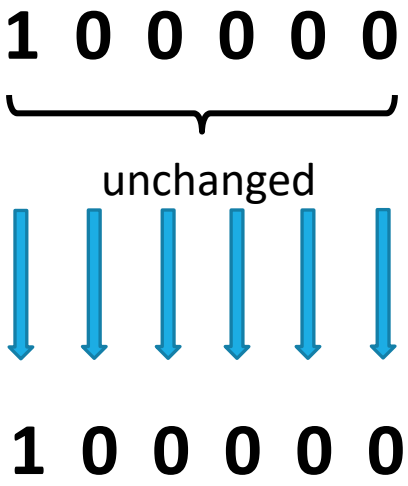
- ✓ **The 2's complement of a binary number can be obtained by leaving the least significant 0's and the first 1 unchanged and then remaining bits, replace 1's with 0's and 0's with 1's.**

- ✓ For example: 2's complement of $(1110100)_2$



2's complement of $(1110100)_2 = (0001100)_2$

2's complement of $(100000)_2$



2's complement of $(100000)_2 = (100000)_2$

RADIX (b's) COMPLEMENT

- ✓ For **decimal number**, it is **10's complement** and can be represented as

$$(10^n - N)$$

- ✓ For Example: 10's complement of $(54230)_{10}$

$$10^5 - (54230)_{10}$$

$$\Rightarrow (100000) - (54230)$$

$$\Rightarrow \ 45770$$

9's complement of $(54230)_{10}$ = $(45769)_{10}$

10's complement of $(54230)_{10}$

= 9's complement of $(54230)_{10}$ + 1

= $(45769)_{10}$ + 1 = $(45770)_{10}$

RADIX (b's) COMPLEMENT

Note:

- ✓ **The 10's complement of a decimal number can be obtained by leaving the least significant 0's unchanged and the first least non-zero element should be subtracted from 10 and other digits from 9.**
- ✓ For example: 10's complement of **$(2860010)_{10}$**

$$2\ 8\ 6\ 0\ 0\ 1\ \underbrace{0}$$

unchanged

$$\begin{array}{r} 9\ 9\ 9\ 9\ 9\ 10\ 0 \\ -\ 2\ 8\ 6\ 0\ 0\ 1 \\ \hline 7\ 1\ 3\ 9\ 9\ 9\ \ 0 \end{array}$$

10's complement of
$(2860010)_{10} = (7139990)_2$

SUBTRACTION WITH COMPLEMENT

- ✓ In digital hardware, complement method for subtraction provides more efficient results compared to borrow concept method.
- ✓ Here, the subtraction is pursued for unsigned numbers using the following method
  - **Subtraction of unsigned numbers with b's Complement**
  - **Subtraction of unsigned numbers (b-1)'s Complement**

SUBTRACTION WITH b's COMPLEMENT

Subtraction of unsigned numbers with b's Complement

✓ The subtraction of **two n-digit unsigned numbers M - N** in base **b** can be done as follows:

1. Add the minuend **M** to the b's complement of the subtrahend **N**. Mathematically,

   $$M + (b^n - N) = M - N + b^n$$

2. If $M \geq N$, the sum will produce an end carry b^n, which can be discarded; what is left is the result **M - N**.

3. If $M < N$, the sum does not produce an end carry and is equal to $b^n - (N - M)$, which is the b's complement of (**N - M**). To obtain the answer in a familiar form, take the b's complement of the sum and place a negative sign in front.

SUBTRACTION WITH b's COMPLEMENT

Example-1:

✓ Using 2's complement, subtract $(101111)_2 - (100110)_2$.

Solution: Here, $M = (101111)_2$ and $N = (100110)_2$

2's complement of $N = (011010)_2$

Step -1: Add M with 2's complement of N

```
  1  1  1  1  1
     1  0  1  1  1  1
+    0  1  1  0  1  0
-----------------------
  (1) 0  0  1  0  0  1
```

Discard End Carry

Step -2: Here M>N, hence, the result is $(101111)_2 - (100110)_2 = (001001)_2$

SUBTRACTION WITH b's COMPLEMENT

Example-2:

✓ Using 2's complement, subtract $(100)_2 - (110000)_2$.

Solution: Here, $M = (100)_2$ and $N = (110000)_2$

2's complement of $N = (010000)_2$

Step -1: Add M with 2's complement of N

```
      0  0  0  1  0  0
+     0  1  0  0  0  0
-----------------------
      0  1  0  1  0  0
```

Step -2: Here M<N, hence, the result is obtained by taking the 2's complement of sum with a negative sign in front.

$$(100)_2 - (110000)_2 = -[\text{2's complement of } (010100)_2] = -(101100)_2$$

SUBTRACTION WITH b's COMPLEMENT

Example-3:

✓ Using 10's complement, subtract **52532 – 3250**.

Solution: Here, **M = 52532** and **N = 3250**

10's complement of **N = (99999 – 03250) + 1**

= 96749 + 1 = 96750

Step -1: Add M with 10's complement of N

```
    1       1
    5   2   5   3   2
+   9   6   7   5   0
-----------------------
(1) 4   9   2   8   2
```

Discard End Carry

Step -2: Here M>N, hence, the result is **52532 – 3250 = 49282**

SUBTRACTION WITH b's COMPLEMENT

Example-4:

✓ Using 10's complement, subtract **1020 – 2056**.

Solution: Here, **M = 1020** and **N = 2056**

10's complement of **N = (9999 – 2056) + 1**

= 7943 + 1 = 7944

Step -1: Add M with 10's complement of N

```
    1  0  2  0
+   7  9  4  4
-----------------
    8  9  6  4
```

Step -2: Here M<N, hence, the result is obtained by taking the 10's complement of sum with a negative sign in front.

1020 – 2056 = –[10's complement of 8964] = –[(9999 – 8964) + 1] = – [1035 + 1] = – 1036

SUBTRACTION WITH (b-1)'s COMPLEMENT

Subtraction of unsigned numbers with (b-1)'s Complement

✓ The subtraction of **two n-digit unsigned numbers M - N** in base **b** can be done as follows:

1. Add the minuend **M** to the (b-1)'s complement of the subtrahend **N**. Mathematically,

$$M + ((b^n - 1) - N) = M - N + b^n - 1$$

2. If $M \geq N$, the sum will produce an end carry b^n. To obtain the result, remove the end carry and add 1 to the sum which also termed as **end around carry**.

3. If $M < N$, the sum does not produce any carry. To obtain the answer in a familiar form, take the (b-1)'s complement of the sum and place a negative sign in front.

SUBTRACTION WITH (b-1)'s COMPLEMENT

Example-1:

✓ Using 1's complement, subtract $(101111)_2 - (100110)_2$.

Solution: Here, $M = (101111)_2$ and $N = (100110)_2$

1's complement of $N = (011001)_2$

Step -1: Add M with 1's complement of N

	1	1	1	1	1	1	
		1	0	1	1	1	1
+		0	1	1	0	0	1
	1	0	0	1	0	0	0
+							1
		0	0	1	0	0	1

End Around Carry

Step -2: Here M>N, hence, the result is $(101111)_2 - (100110)_2 = (001001)_2$

SUBTRACTION WITH (b-1)'s COMPLEMENT

Example-2:

✓ Using 1's complement, subtract $(100)_2 - (110000)_2$.

Solution: Here, $M = (100)_2$ and $N = (110000)_2$

1's complement of $N = (001111)_2$

Step -1: Add M with 1's complement of N

		1	1			
	0	0	0	1	0	0
+	0	0	1	1	1	1
	0	1	0	0	1	1

Step -2: Here M<N, hence, the result is obtained by taking the 1's complement of sum with a negative sign in front.

$$(100)_2 - (110000)_2 = -[\text{1's complement of } (010011)_2] = -(101100)_2$$

SUBTRACTION WITH (b-1)'s COMPLEMENT

Example-4:

✓ Using 9's complement, subtract **1020 – 2056**.

Solution: Here, **M = 1020** and **N = 2056**

9's complement of **N = (9999 – 2056)**

= 7943

Step -1: Add M with 9's complement of N

```
    1   0   2   0
+   7   9   4   3
-----------------
    8   9   6   3
```

Step -2: Here M<N, hence, the result is obtained by taking the 9's complement of sum with a negative sign in front.

1020 – 2056 = –[9's complement of 8963] = –(9999 – 8963) = – 1036

Here's why 2's complement works so elegantly to represent negative numbers:

1. Utilizing the Range of Numbers:

- In an n-bit binary system, we have 2^n unique bit patterns. We want to use that entire range to represent both positive and negative numbers.
- 2's complement ingeniously splits the number range in half:
  - The first half (MSB = 0) represents positive numbers (including zero).
  - The second half (MSB = 1) cleverly maps to negative numbers.

The Magic of Modular Arithmetic

- Think of the binary numbers in a fixed width (e.g., 8 bits) as forming a 'circle' or a modular system. Since we only have 8 bits, if you increase beyond the maximum positive value, you wrap around to the negative side.
- **Example (8-bit system):**
  - 127 (0111 1111) is the largest positive number.
  - If you add 1, you'd expect 128, but with 8 bits, it wraps around to -128 (1000 0000), the most negative number.
- 2's complement aligns perfectly with this modular 'wrapping' behavior.

The process of taking the 2's complement of a positive number essentially calculates how far you need to "go back" around the circle to reach its negative counterpart.

The Magic of Modular Arithmetic

The beauty is that normal addition and subtraction operations just work. Because of the 2's complement representation, calculations 'wrap around' correctly on the number circle.
Example: 5 - 3 translates to adding 5 + (-3) in 2's complement.
Let's break down the "5 - 3" example to make this concrete (using 8-bit representation):

5 in binary: 0000 0101
-3 in 2's complement: 1111 1101
Adding them together:

```
    0000 0101
+   1111 1101
    ----------
 1 0000 0010
```

Due to the bit limit, the leading 1 (overflow) is discarded, leaving us with 0000 0010, which is the binary representation of 2.

In essence, 2's complement leverages the inherent behavior of modular arithmetic within a fixed binary system to create an incredibly efficient and consistent representation of negative numbers.

SIGNED BINARY NUMBERS

- ✓ Hence, a –ve number can be represented in 3 ways
  - **Signed Magnitude Representation**
  - **1's Complement Representation**
  - **2's Complement Representation**
- ✓ For example :
  - **– 7** Representation

    Signed Magnitude Representation

    1 111

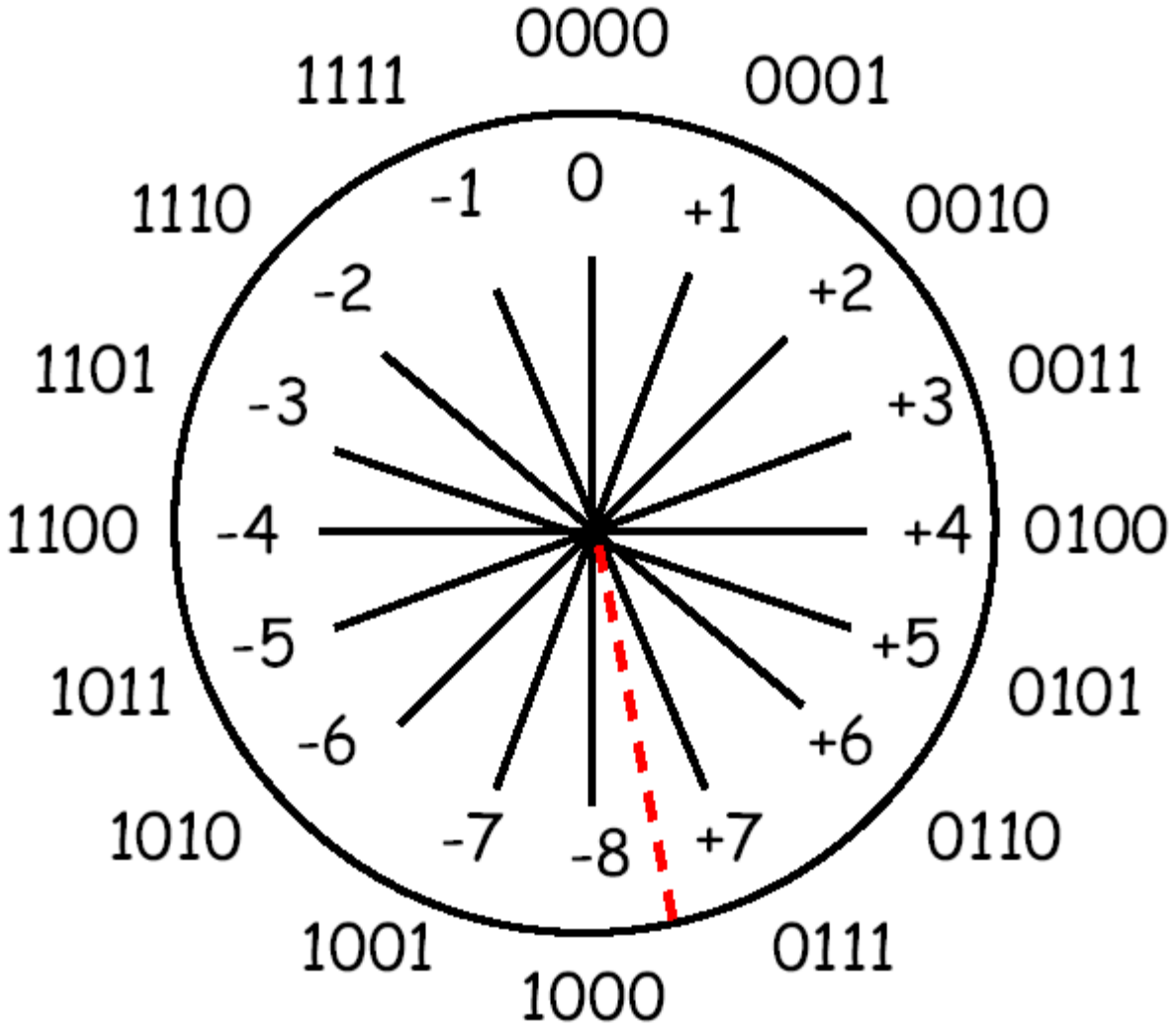
    1's Complement Representation

    1 000

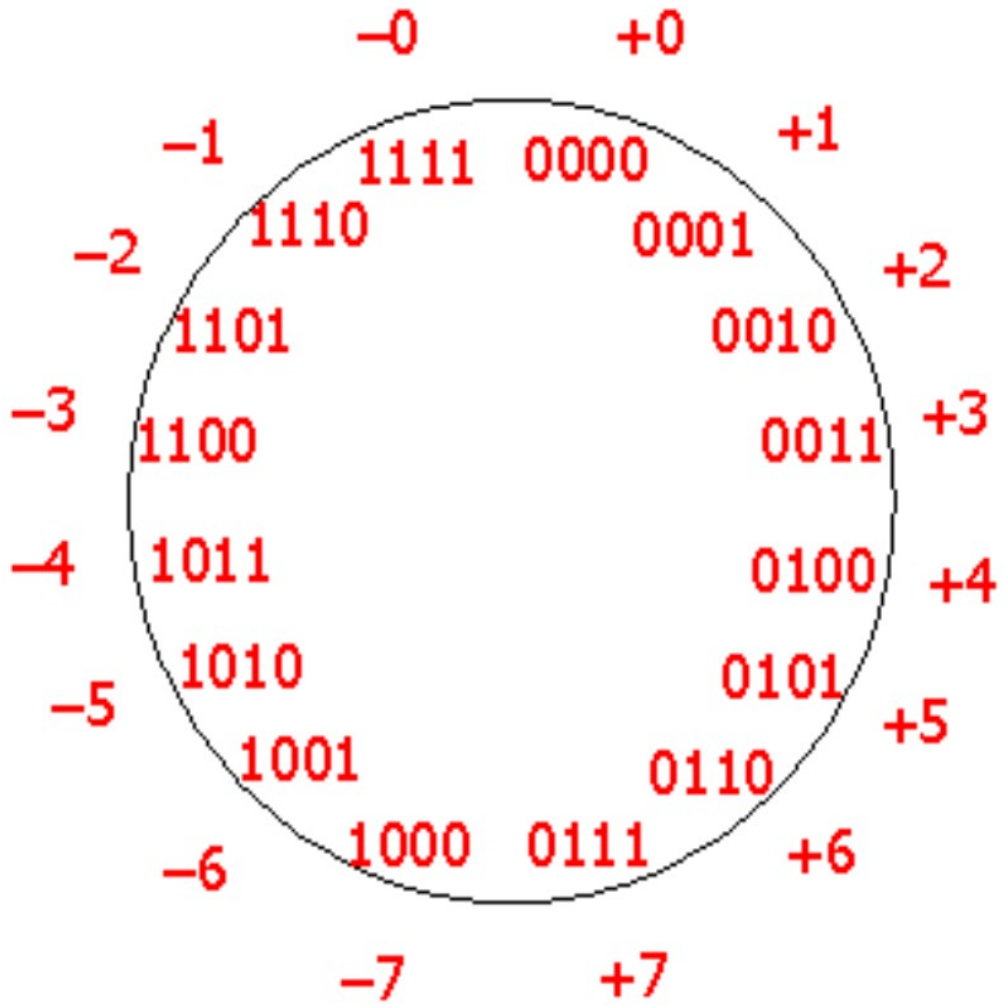
    2's Complement Representation

    1 001

id	b_3	b_2	b_1	b_0	Signed	One's	Two's
0	0	0	0	0	0	0	0
1	0	0	0	1	1	1	1
2	0	0	1	0	2	2	2
3	0	0	1	1	3	3	3
4	0	1	0	0	4	4	4
5	0	1	0	1	5	5	5
6	0	1	1	0	6	6	6
7	0	1	1	1	7	7	7
8	1	0	0	0	-0	-7	-8
9	1	0	0	1	-1	-6	-7
10	1	0	1	0	-2	-5	-6
11	1	0	1	1	-3	-4	-5
12	1	1	0	0	-4	-3	-4
13	1	1	0	1	-5	-2	-3
14	1	1	1	0	-6	-1	-2
15	1	1	1	1	-7	-0	-1



2’s complement



1’s complement number

How they came?

By considering 4 bit 2’s complement number system, based on the formula...

-1 means..... 16-1=15 (1111)

-2 means... 16-2=14 (1110)

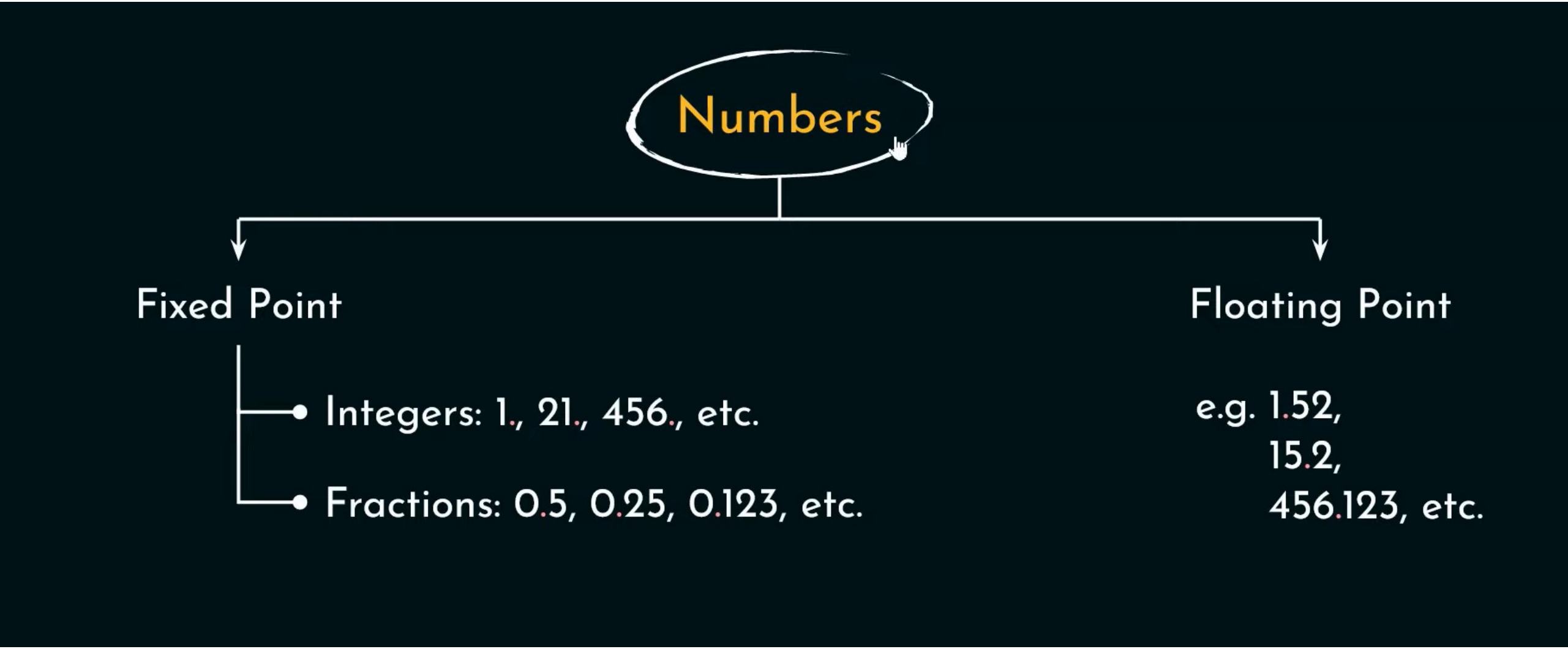
-3 means... 16-3=13 (1101)

ASSIGNMENT QUESTIONS

1. Using 1's, 2's, 9's and 10's complement perform the following

   (a) 23-56

   (b) 98-34


Numbers
Fixed Point
Integers: 1., 21., 456., etc.
Fractions: 0.5, 0.25, 0.123, etc.
Floating Point
e.g. 1.52,
15.2,
456.123, etc.

Decimal Number System

Symbols: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

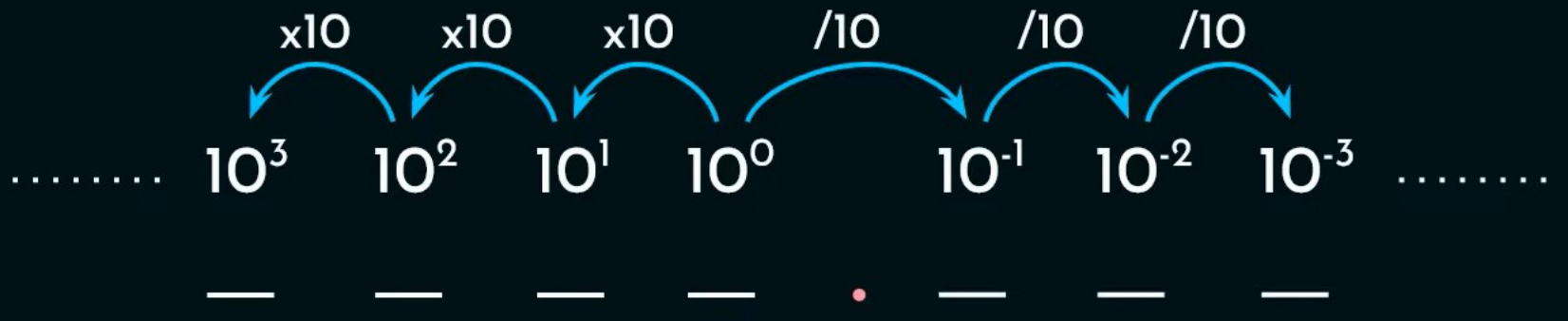
Base: 10

..... Th H T U

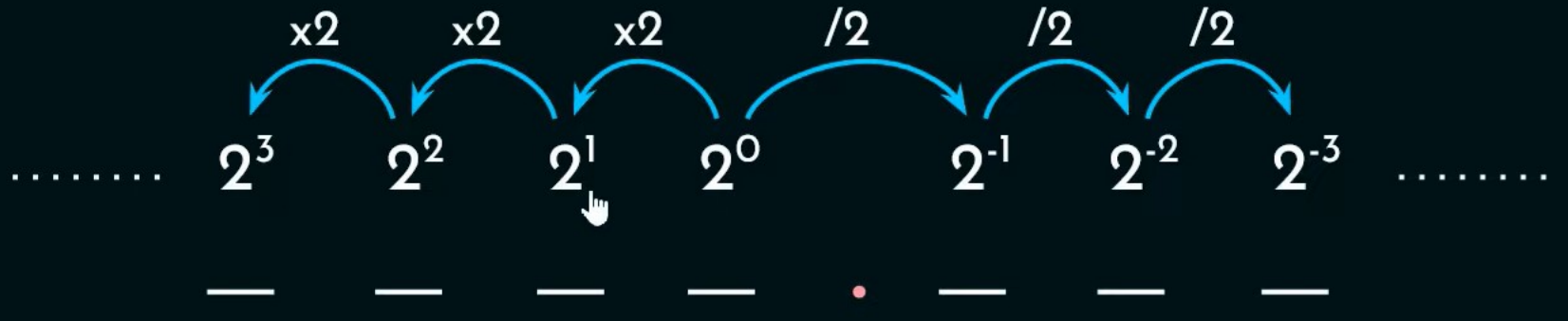
..... 10^3 10^2 10^1 10^0

x10 x10 x10

Decimal Number System

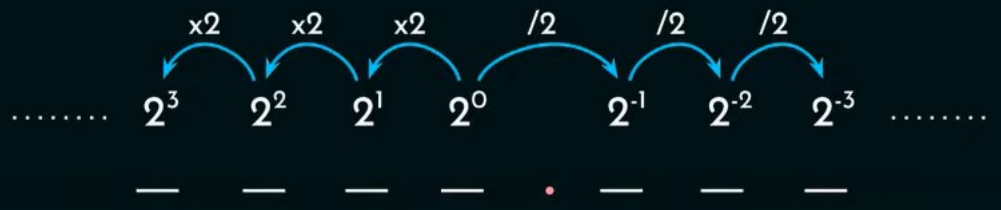


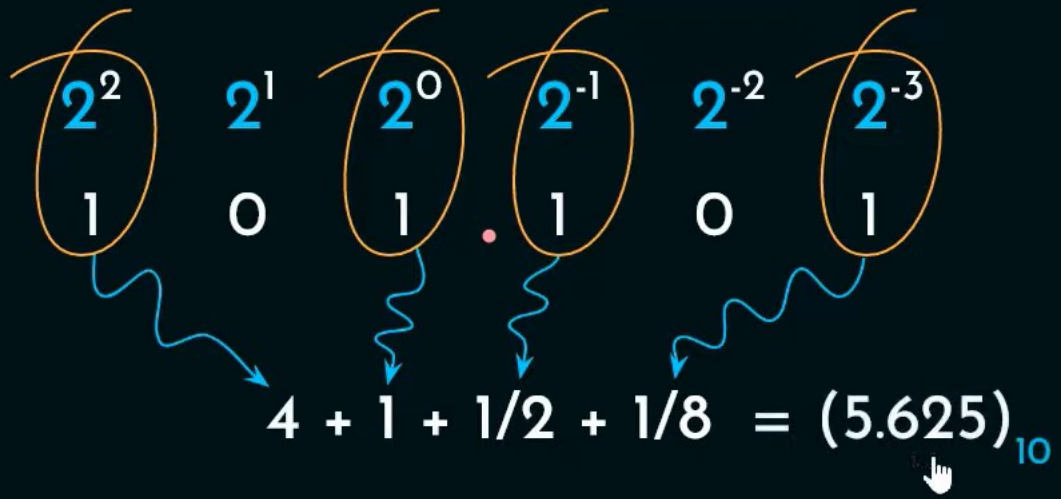
Binary Number System



Binary to Decimal:

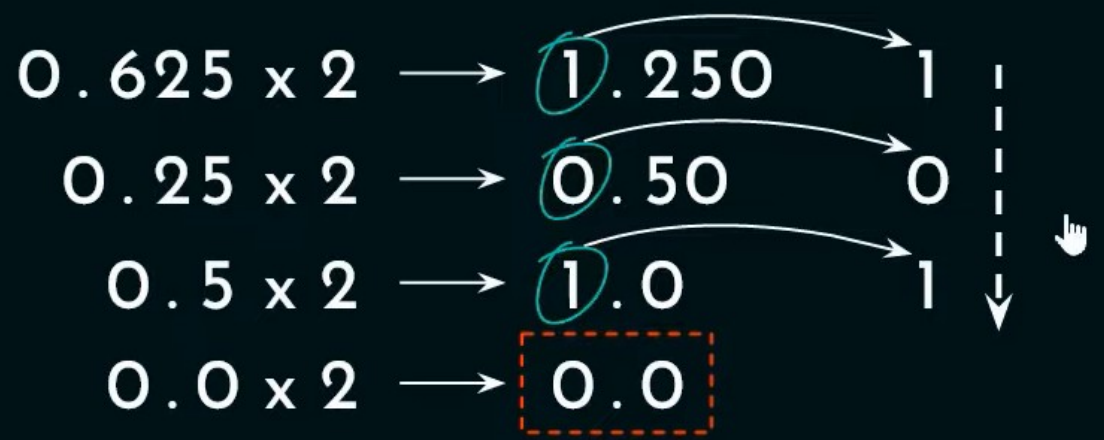
🔍 $(101.101)_2 \rightarrow (?)_{10}$





Decimal to Binary:

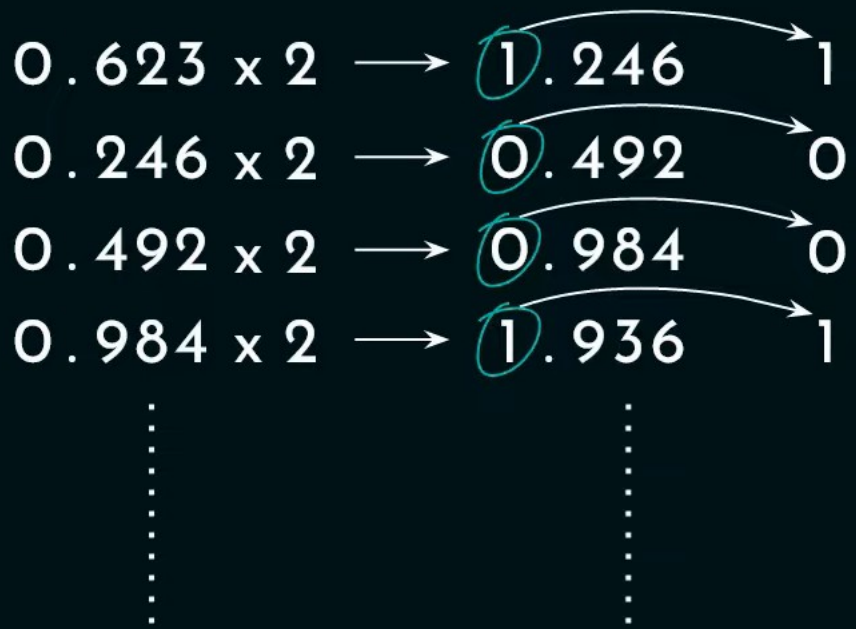
📌 $(0.625)_{10} \rightarrow (0.101)_2$



Decimal to Binary:

📌 $(0.625)_{10} \rightarrow (0.101)_2$

🔍 $(0.623)_{10} \rightarrow (?)_2$



Close approximation using fixed length memory space.

Floating point representations:

📌 $(5.625)_{10} \rightarrow (101.101)_2$

$[101101,3]$

Radix point is present after the 3rd bit from MSb.

This method is considered impractical

Floating point representations:

📌 $(5.625)_{10} \rightarrow (101.101)_2$

$(5.625)_{10} \rightarrow 0.5625 \times 10^{1}$ Exponent

Mantissa

$(101.101)_2 \rightarrow 0.101101 \times 2^{3}$ Exponent

Mantissa

Need of Normalization:

$(101.101)_2 \rightarrow 0.101101 \times 2^3$

$(101.101)_2 \rightarrow 1.01101 \times 2^2$

$(101.101)_2 \rightarrow 0.0101101 \times 2^4$

$(101.101)_2 \rightarrow 101101 \times 2^{-3}$

Representing same number with different formats will bring confusion. Computer can use only one single format.

Normalization:

-- Two types.

1. Explicit Normalization: Move the radix point to the LHS of the most significant '1' in the bit sequence.

   $(101.101)_2 \rightarrow 0.101101 \times 2^3$

✓ 2. Implicit Normalization: Move the radix point to the RHS of the most significant '1' in the bit sequence.

   $(101.101)_2 \rightarrow 1.01101 \times 2^2$

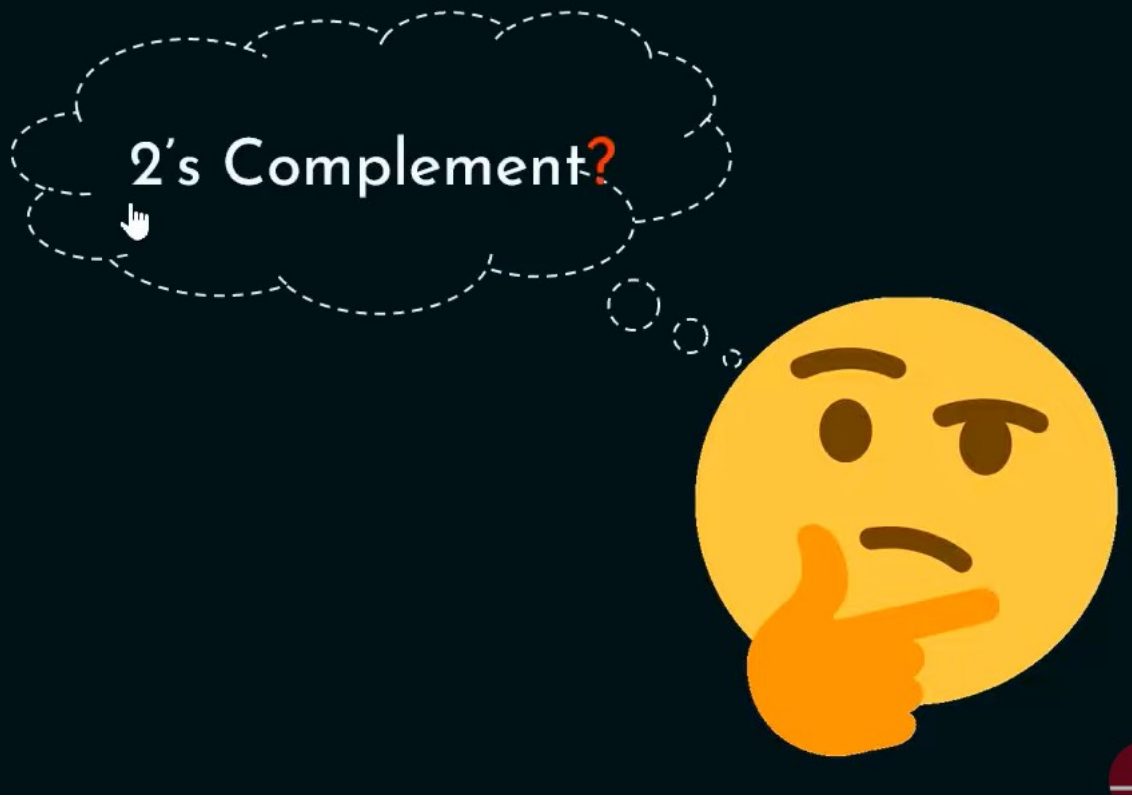
Implicit is considered to be better

Biasing:

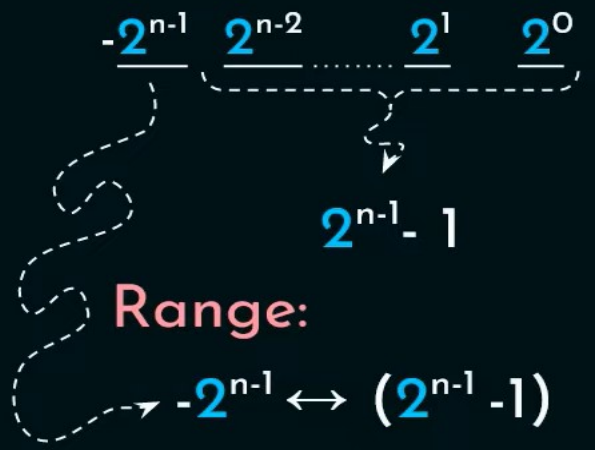
S	Exponent	Mantissa

$(101.101)_2 \rightarrow 0.101101 \times 2^3$

$(101.101)_2 \rightarrow 101101 \times 2^{-3}$



Signed numbers Representations:

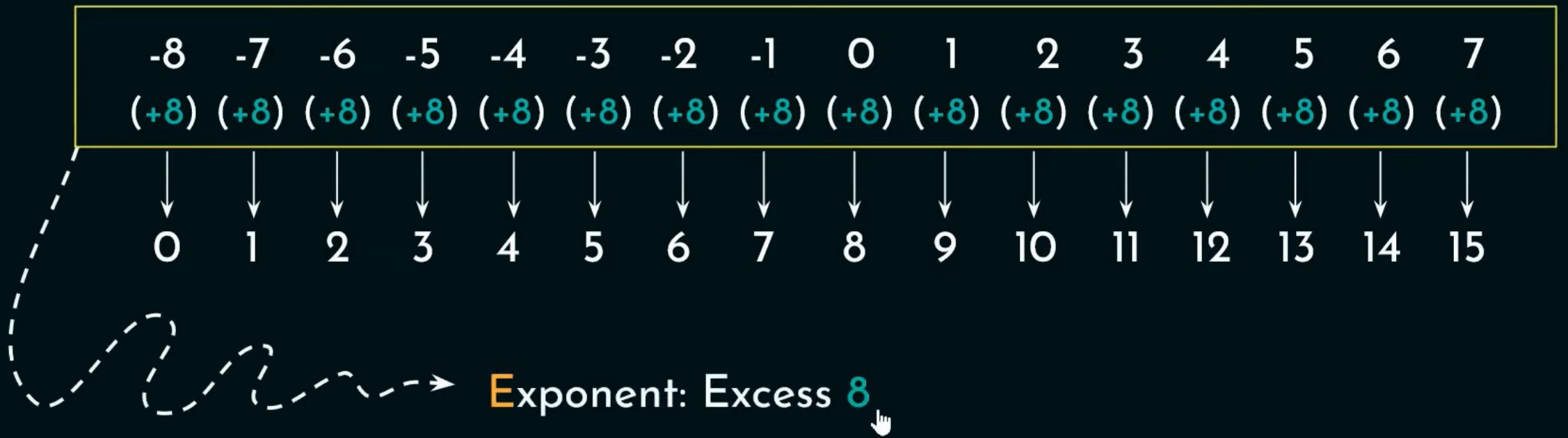


-2^3	2^2	2^1	2^0	2's	
0	0	0	0	0	-8
0	0	0	1	1	-7
0	0	1	0	2	-6
0	0	1	1	3	-5
0	1	0	0	4	-4
0	1	0	1	5	-3
0	1	1	0	6	-2
0	1	1	1	7	-1
1	0	0	0	-8	0
1	0	0	1	-7	1
1	0	1	0	-6	2
1	0	1	1	-5	3
1	1	0	0	-4	4
1	1	0	1	-3	5
1	1	1	0	-2	6
1	1	1	1	-1	7

The numbers are not in sequence!!!!!! So, we go for biasing.

Biasing:


S
Exponent
Mantissa


-8 -7 -6 -5 -4 -3 -2 -1 0 1 2 3 4 5 6 7
(+8) (+8) (+8) (+8) (+8) (+8) (+8) (+8) (+8) (+8) (+8) (+8) (+8) (+8) (+8) (+8)
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
Exponent: Excess 8

Biasing:

$(0.0101)_2 \rightarrow 0.101 \times 2^{-1}$

$(101.101)_2 \rightarrow 1.01101 \times 2^{2}$

-8	-7	-6	-5	-4	-3	-2	-1	0	1	2	3	4	5	6	7
(+8)	(+8)	(+8)	(+8)	(+8)	(+8)	(+8)	(+8)	(+8)	(+8)	(+8)	(+8)	(+8)	(+8)	(+8)	(+8)
↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

$(0.0101)_2 \rightarrow$

S	Exponent				Mantissa				
0	0	1	1	1	1	0	1	0	0

$(101.101)_2 \rightarrow$

S	Exponent				Mantissa				
0	1	0	1	0	0	1	1	0	1

EXPLICIT VS IMPLICIT

Formula:

1. Explicit Normalization: $(-1)^{S} \times 0.M \times 2^{E\text{-}Bias}$
2. Implicit Normalization: $(-1)^{S} \times 1.M \times 2^{E\text{-}Bias}$

Getting back from the memory into readable form!!!

Explicit vs. Implicit Normalization:

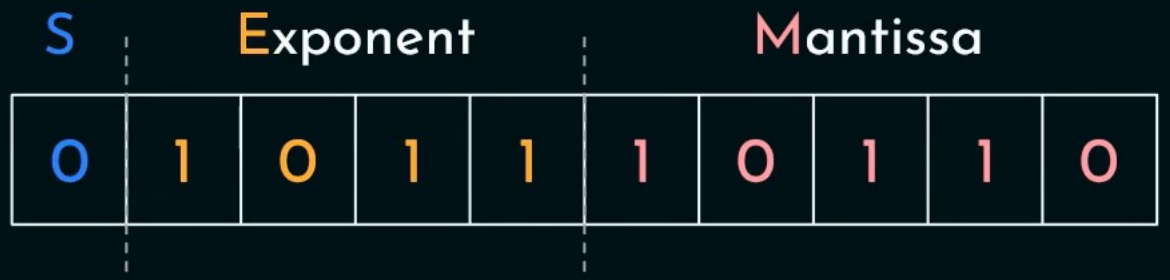
$(5.625)_{10} \rightarrow (101.101)_2$

$(101.101)_2 \rightarrow 0.101101 \times 2^3$

S = 0

E = 3 + 8 = 11 $(1011)_2$

M = 10110

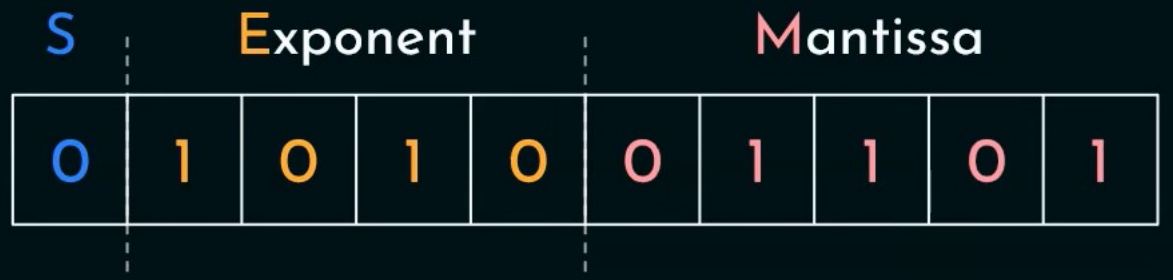


$(101.101)_2 \rightarrow 1.01101 \times 2^2$

S = 0

E = 2 + 8 = 10 $(1010)_2$

M = 01101

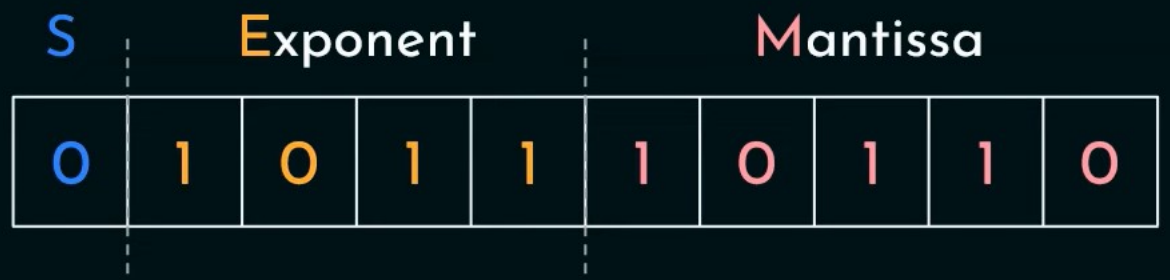


We are loosing bits in explicit

Explicit vs. Implicit Normalization:

📌 $(5.625)_{10} \rightarrow (101.101)_2$

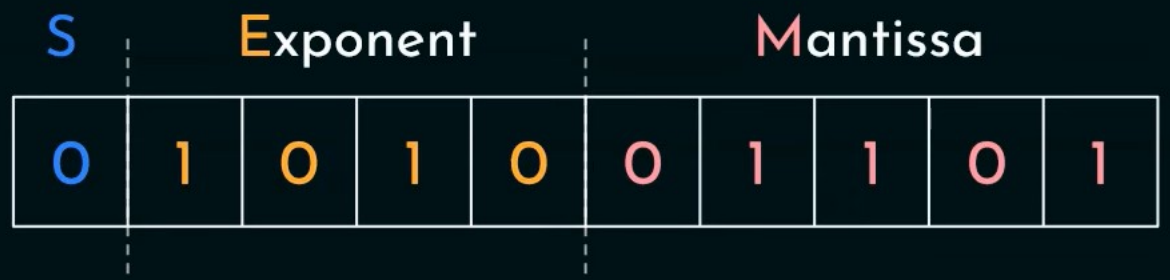
🔍 $(101.101)_2 \rightarrow 0.101101 \times 2^3$



$(-1)^S \times 0.M \times 2^{E\text{-Bias}}$

$(101.1)_2 \rightarrow (5.5)_{10}$

🔍 $(101.101)_2 \rightarrow 1.01101 \times 2^2$



$(-1)^S \times 1.M \times 2^{E\text{-Bias}}$

$(101.101)_2 \rightarrow (5.625)_{10}$

More precision in implicit normalization

IEEE 754:

- IEEE 754-1985.
- IEEE 754-2008.
- IEEE 754-2019.

IEEE 754:

Name	Common Name	Significand bits	Exponent bits	Exponent Bias	Organization of Bits
binary16	Half precision	11	5	15	
binary32	Single precision	24	8	127	
binary64	Double precision	53	11	1023	
binary128	Quadruple precision	113	15	16383	
binary256	Octuple precision	237	19	262143	

IEEE 754 – Single precision:

S(1 bit)	E(8 bits)	M(23 bits)	Representation	
0 or 1	0000 0000 (E = 0)	0000. . . . . 0 (M = 0)	±0	
0 or 1	1111 1111 (E = 255)	0000. . . . . 0 (M = 0)	±∞	
0 or 1	1 ⩽ E ⩽ 254	M= xxxx. x	Implicit Normalized Form	$(-1)^S \times 1.M \times 2^{E-127}$
0 or 1	E = 0	M ≠ 0	Fractional Form	$(-1)^S \times 0.M \times 2^{-126}$
	E = 255	M ≠ 0	NAN	

IEEE 754 – Double precision:

S(1 bit)	E(11 bits)	M(52 bits)	Representation	
0 or 1	000 0000 0000 (E = 0)	0000. . . . . 0 (M = 0)	±0	
0 or 1	111 1111 1111 (E = 2047)	0000. . . . . 0 (M = 0)	±∞	
0 or 1	1 ⩽ E ⩽ 2046	M= xxxx. x	Implicit Normalized Form	$(-1)^S \times 1.M \times 2^{E-1023}$
0 or 1	E = 0	M ≠ 0	Fractional Form	$(-1)^S \times 0.M \times 2^{-1022}$
	E = 2047	M ≠ 0	NAN	

Problems on Floating point numbers

Q: Suppose a 16 bit register of the following format is used for storing binary floating point numbers:
Mantissa(M) is denoted using normalized Sign-magnitude fraction.
Exponent(E) is expressed in excess-64 form.

1. How many bits are used for the fractional mantissa?
2. What should be the expression for the decimal value?
3. What is largest number (in Base 10) that can be represented using this register?
4. What should be the bit patterns for $(7.5)_{10}$ & $(-16.125)_{10}$?
5. Determine the difference between the smallest positive number and the 2nd smallest positive number in this representation?

Q: Suppose a 16 bit register of the following format is used for storing binary floating point numbers:
Mantissa(M) is denoted using normalized Sign-magnitude fraction.
Exponent(E) is expressed in excess-64 form.

1. How many bits are used for the fractional mantissa?

Sol. Excess-64 bias is used. ∴ Maximum -ve value = $-64 = -2^{6} = -2^{7-1}$

Q: Suppose a 16 bit register of the following format is used for storing binary floating point numbers:
Mantissa(M) is denoted using normalized Sign-magnitude fraction.
Exponent(E) is expressed in excess-64 form.

2. What should be the expression for the decimal value?

Sol. $(-1)^S \times 1.M \times 2^{E\text{-Bias}} \Rightarrow (-1)^S \times 1.M \times 2^{E-64}$

Q: Suppose a 16 bit register of the following format is used for storing binary floating point numbers:
Mantissa(M) is denoted using normalized Sign-magnitude fraction.
Exponent(E) is expressed in excess-64 form.

3. What is largest number (in Base 10) that can be represented using this register?

Sol. $(-1)^S \times 1.M \times 2^{E-64} \Rightarrow (-1)^0 \times 1.11111111 \times 2^{127-64}$

$\Rightarrow 1.11111111 \times 2^{63}$

$\Rightarrow 111111111 \times 2^{55}$

$0011111111100000 \times 10^{15}$

3 F E 0

$3 \times 16^3 + 15 \times 16^2 + 14 \times 16^1 + 0 \times 16^0 \longrightarrow 16352$

$2^{10} \cong 10^3$

Q: Suppose a 16 bit register of the following format is used for storing binary floating point numbers:
Mantissa(M) is denoted using normalized Sign-magnitude fraction.
Exponent(E) is expressed in excess-64 form.

4. What should be the bit patterns for $(7.5)_{10}$ & $(-16.125)_{10}$?

Sol. $(7.5)_{10} \rightarrow (111.1)_2$

$\Rightarrow 1.111 \times 2^2$

$E = 2 + 64 = 66$

$(-16.125)_{10} \rightarrow (10000.001)_2$

$\Rightarrow 1.0000001 \times 2^4$

$E = 4 + 64 = 68$

Q: Suppose a 16 bit register of the following format is used for storing binary floating point numbers:
Mantissa(M) is denoted using normalized Sign-magnitude fraction.
Exponent(E) is expressed in excess-64 form.

5. Determine the difference between the smallest positive number and the 2nd smallest positive number in this representation?

Sol. $(-1)^S \times 1.M \times 2^{E-64}$

Smallest +ve number:

$(-1)^0 \times 1.00000000 \times 2^{0-64} \Rightarrow 1.0 \times 2^{-64}$

2nd smallest +ve number:

$(-1)^0 \times 1.00000001 \times 2^{0-64} \Rightarrow 1.00000001 \times 2^{-64}$

Binary Multiplication:

Multiplicand(M): 1 0 1 0 Multiplier(Q): 0 1 1 0

			1	0	1	0
		x	0	1	1	0
			0	0	0	0
		1	0	1	0	x
	1	0	1	0	x	
0	0	0	0	x		
0	1	1	1	1	0	0

Binary Multiplication – Partial Sum Approach:

|M| : n-bits |Q| : n-bits |M x Q| : (n+n) = 2n-bits

Multiplicand(M): 1 1 1 1 Multiplier(Q): 1 1 1 1

			1	1	1	1	
		x	1	1	1	1	
			1	1	1	1	
		1	1	1	1	x	
	1	1	1	1	x		
1	1	1	1	x			
1	1	1	0	0	0	0	1

Binary Multiplication – Partial Sum Approach:

For every bit in Q(from right to left),

i. If the bit is 0, perform right shift once on the register's content.
ii. If the bit is 1, first add M with the most significant n-bits of the register's content then perform right shift once.

Multiplicand(M): 1 0 1 0

Multiplier(Q): 0 1 1 0

#Right shifts: n

#Additions: #1s in Q

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
+ 1	0	1	0				
1	0	1	0	0	0	0	0
0	1	0	1	0	0	0	0
+ 1	0	1	0				
1	1	1	1	0	0	0	0
0	1	1	1	1	0	0	0
0	0	1	1	1	1	0	0

Binary Multiplication – Booth's Algorithm:

- Multiplies two signed binary numbers in two's complement notation.
- Invented by Andrew Donald Booth in 1950.

Binary Multiplication – Booth's Algorithm:

Multiplier(Q): 0 0 1 1 1 1 1 0 ⟶ 32 + 16 + 8 + 4 + 2 = 62

2^7 2^6 2^5 2^4 2^3 2^2 2^1 2^0

Binary Multiplication – Booth's Algorithm:

Multiplier(Q): 0 0 1 1 1 1 1 0 $\longrightarrow$ $2^{5+1} - 2^1 = 64 - 2$

$2^7\ 2^6\ 2^5\ 2^4\ 2^3\ 2^2\ 2^1\ 2^0$ $= 62$

0 1 1 0 1 1 1 0 $\longrightarrow$ $(2^7 - 2^5) + (2^4 - 2^1)$

$2^7\ 2^6\ 2^5\ 2^4\ 2^3\ 2^2\ 2^1\ 2^0$ $= 96 + 14 = 110$

Generalization: 0 0 0 1 1 1 1 . . . . . 1 0 0 . . . 0

(2^j … 2^i)

$2^{j+1} - 2^i$

Binary Multiplication – Booth's Algorithm:

Multiplicand(M): 0 1 0 1 1 ($\overline{M}$): 1 0 1 0 1

Multiplier(Q): 0 1 1 1 0 ⟶ $2^4 - 2^1$

2^4 2^3 2^2 2^1 2^0

M x Q: $M \times (2^4 - 2^1) = 2^4(M) + 2^1(\overline{M})$

Binary Multiplication – Booth's Algorithm:

Multiplicand(M): 0 1 0 1 1 $(\overline{M})$: 1 0 1 0 1

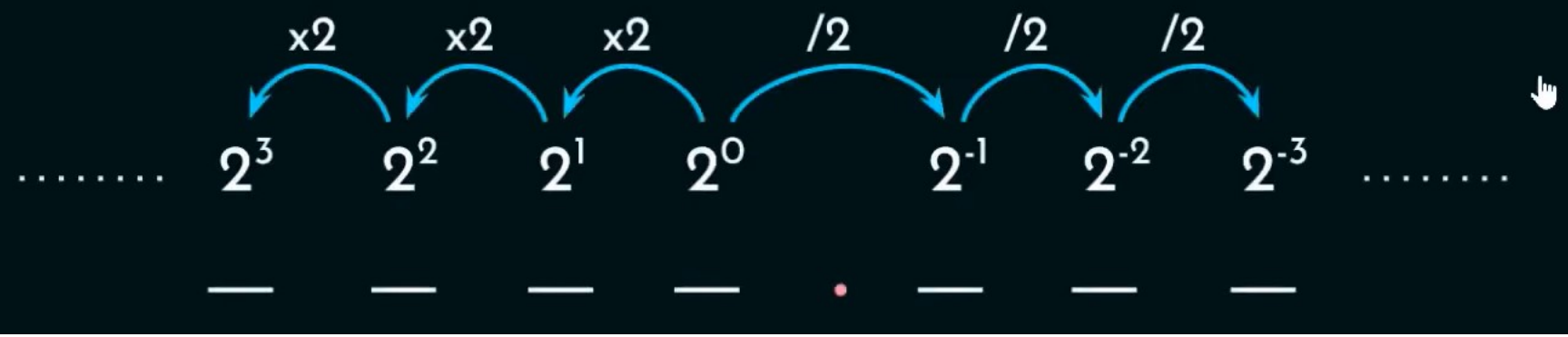
Multiplier(Q): 0 1 1 1 0 ⟶ $2^4 - 2^1$

2^4 2^3 2^2 2^1 2^0

M x Q: $2^4(M) + 2^1(\overline{M})$

$2^4(M)$: 0 1 0 1 1 x 2^4 = 0 1 0 1 1 0 0 0 0

$2^1(\overline{M})$: 1 0 1 0 1 x 2^1 = 1 0 1 0 1 0



Binary Multiplication – Booth's Algorithm:

Multiplicand(M): 0 1 0 1 1 $(\overline{M})$: 1 0 1 0 1

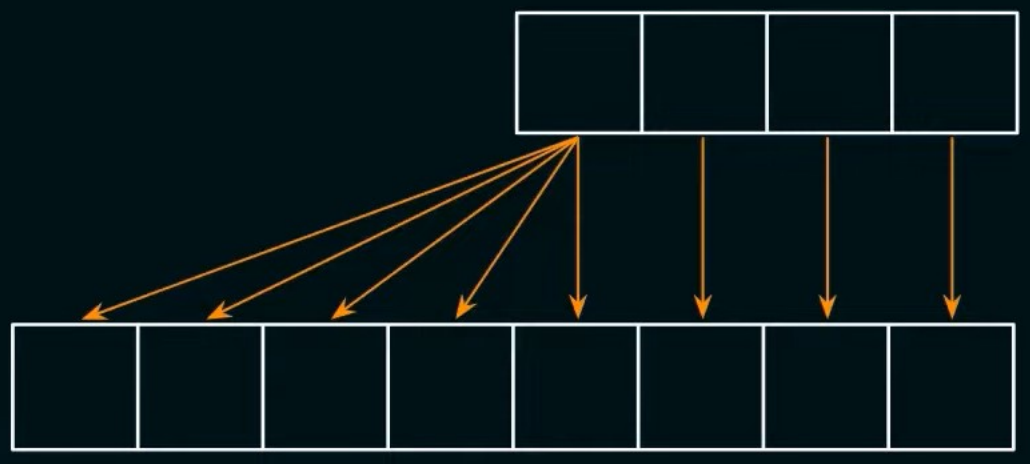
Multiplier(Q): 0 1 1 1 0 ⟶ $2^4 - 2^1$

2^4 2^3 2^2 2^1 2^0

M x Q: $2^4(M) + 2^1(\overline{M})$

$2^4(M)$: 0 1 0 1 1 x 2^4 = 0 1 0 1 1 0 0 0 0

$2^1(\overline{M})$: 1 0 1 0 1 x 2^1 = 1 1 1 1 0 1 0 1 0



Copy the contents at the Least Significant Places & copy the MSb to the rest of the cells.

Binary Multiplication – Booth's Algorithm:

Multiplicand(M): 0 1 0 1 1 ($\overline{M}$): 1 0 1 0 1 → 1's Complement of M + 1

Multiplier(Q): 0 1 1 1 0 → $2^4 - 2^1$

2^4 2^3 2^2 2^1 2^0

M x Q: $2^4(M) + 2^1(\overline{M})$

$2^4(M)$: 0 1 0 1 1 0 0 0 0

$2^1(\overline{M})$: + 1 1 1 1 0 1 0 1 0 → 5 Shifts

1 0 1 0 0 1 1 0 1 0

With the Booths algorithm, the number of additions are reduced. So efficient….

Binary Multiplication – Booth's Algorithm:

M x Q: $2^i(M) + 2^i(\overline{M})$

Multiplicand(M): 0 1 0 1 1 

Multiplier(Q): 0 1 1 1 0

Binary Multiplication – Booth's Algorithm:

$M \times Q$: $2^i(M) + 2^i(\overline{M})$

Multiplicand(M): 0 1 0 1 1 

Multiplier(Q): 0 1 1 1 0

Q_{LSb}	Q_{-1}	Operation
0	0	Arithmetic Right Shift
0	1	Acc ← Acc + M, ARS
1	0	Acc ← Acc + $\overline{M}$, ARS
1	1	Arithmetic Right Shift

Binary Multiplication – Booth's Algorithm:

$M \times Q$: $2^i(M) + 2^i(\overline{M})$

Multiplicand(M): 0 1 0 1 1 

Multiplier(Q): 0 1 1 1 0

Q_{LSb}	Q_{-1}	Operation
0	0	Arithmetic Right Shift
0	1	Acc ← Acc + M, ARS
1	0	Acc ← Acc + $\overline{M}$, ARS
1	1	Arithmetic Right Shift

Accumulator	Q	Q_{-1}
0 0 0 0 0	0 1 1 1 0	0
0 0 0 0 0	0 0 1 1 1	0
+ 1 0 1 0 1		
1 0 1 0 1	0 0 1 1 1	0
1 1 0 1 0	1 0 0 1 1	1
1 1 1 0 1	0 1 0 0 1	1
1 1 1 1 0	1 0 1 0 0	1
+ 0 1 0 1 1		
1 0 1 0 0 1	1 0 1 0 0	1
0 0 1 0 0	1 1 0 1 0	0

Binary Multiplication – Booth's Algorithm:

$M \times Q$: $2^i(M) + 2^i(\overline{M})$

Multiplicand(M): 0 1 0 1 1 

Multiplier(Q): 0 1 1 1 0

$M \times Q$: $2^4(M) + 2^1(\overline{M})$

$2^4(M)$: 0 1 0 1 1 0 0 0 0

$2^1(\overline{M})$: + 1 1 1 1 0 1 0 1 0

1 0 1 0 0 1 1 0 1 0

	Accumulator	Q	Q_{-1}
	0 0 0 0 0	0 1 1 1 0	0
	0 0 0 0 0	0 0 1 1 1	0
+	1 0 1 0 1		
	1 0 1 0 1	0 0 1 1 1	0
	1 1 0 1 0	1 0 0 1 1	1
	1 1 1 0 1	0 1 0 0 1	1
	1 1 1 1 0	1 0 1 0 0	1
+	0 1 0 1 1		
	1 0 1 0 0 1	1 0 1 0 0	1
	0 0 1 0 0	1 1 0 1 0	0

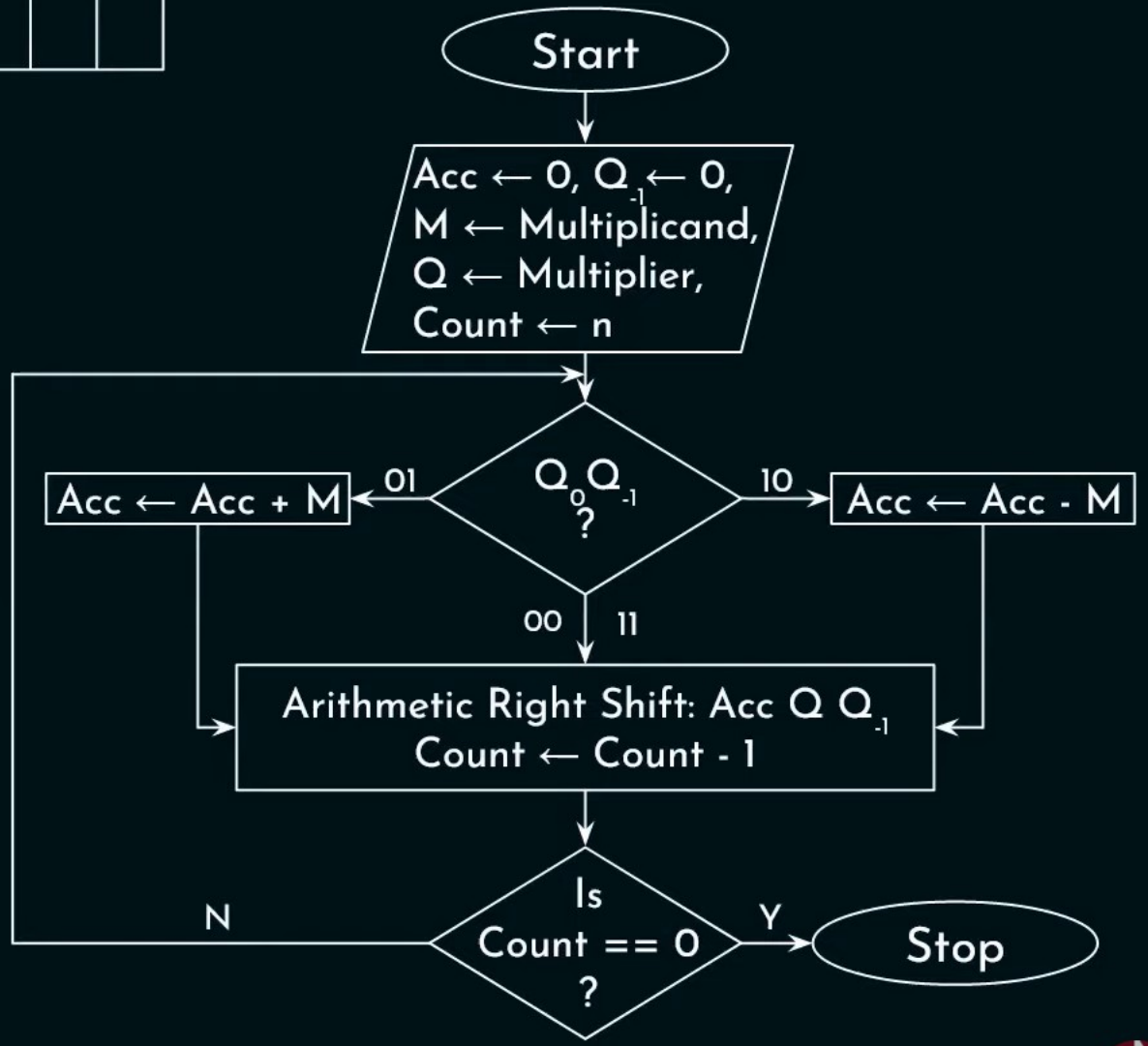
Binary Multiplication – Booth's Algorithm:

$M \times Q$: $2^j(M) + 2^i(\overline{M})$

Multiplicand(M): 0 1 0 1 1

Multiplier(Q): 0 1 1 1 0

Q_{LSb}	Q_{-1}	Operation
0	0	Arithmetic Right Shift
0	1	Acc ← Acc + M, ARS
1	0	Acc ← Acc + $\overline{M}$, ARS
1	1	Arithmetic Right Shift



Binary Multiplication – Booth's Algorithm:

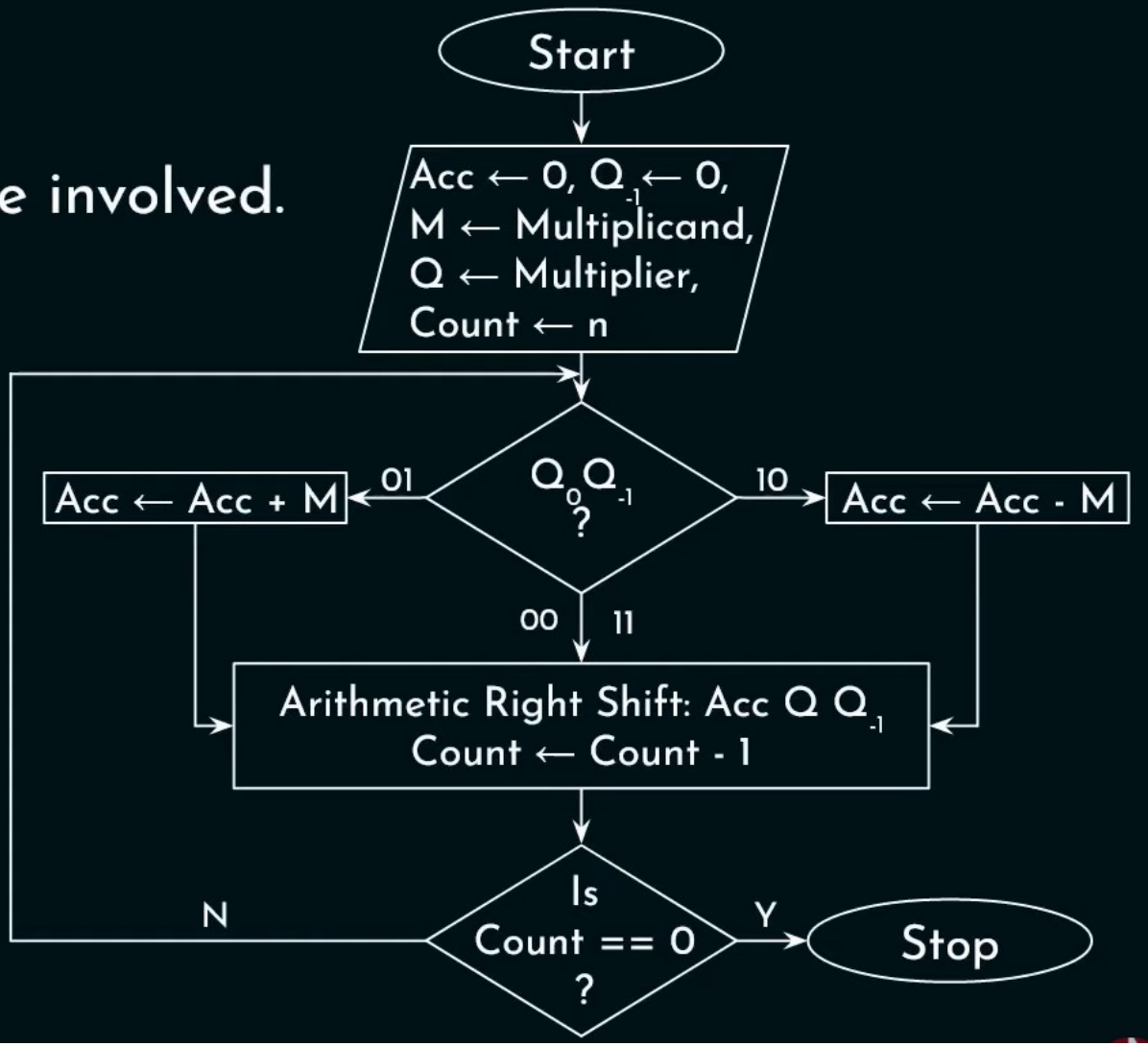
M x Q: $2^i(M) + 2^i(\overline{M})$

Advantages:

1. Performs signed multiplications.
2. Performs better if -ve numbers are involved.
   e.g. 1111111111001 → -7
3. Reduced number of operations.

Disadvantage:

Performance decreases,
if Q: 01010101010....



Binary Division:

Dividend(d): $1\ 0\ 1\ 0\ 1\ 0 = (42)_{10}$

Divisor(v): $1\ 0\ 1\ 1 = (11)_{10}$

Quotient(q): $1\ 1 = (3)_{10}$

Remainder(r): $1\ 0\ 0\ 1$

```
          0 1 1
      ┌─────────
1 0 1 1 │ 1 0 1 0 1 0
        - 1 0 1 1
        ─────────
          1 0 1 0 0
        - 1 0 1 1
        ─────────
            1 0 0 1
```

```
        3
    ┌─────
1 1 │ 4 2
    - 3 3
    ─────
        9
```

Division:

-- Can be grouped into 2 classes.

Slow Algorithms
(Iterative operation: Subtraction)

-- Restoring Division

-- Non-Restoring Division

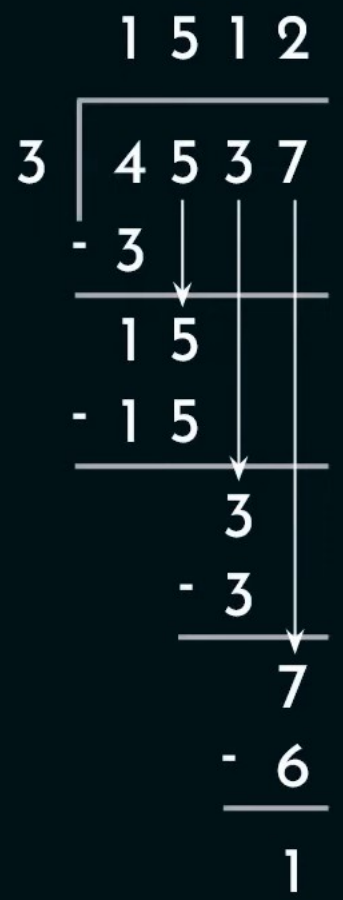
Fast Algorithms
(Iterative operation: Multiplication)

-- Division by Series' Expansion

-- Newton-Raphson Division

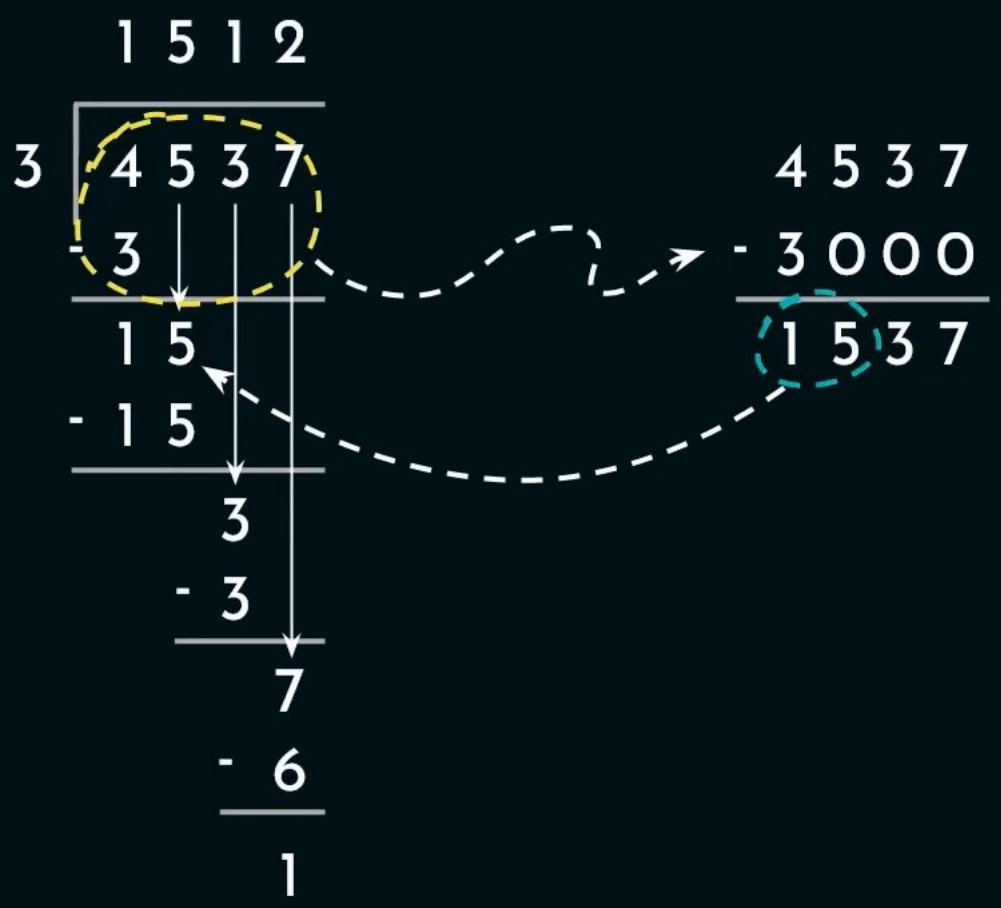
Restoring Division:

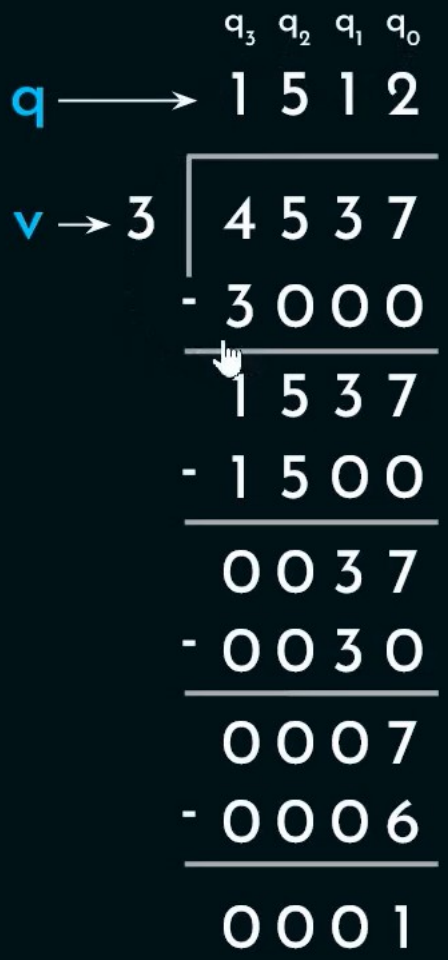
Dividend (d): 4 5 3 7 Divisor (v): 3 Quotient (q): 1 5 1 2 Remainder (r): 1



Restoring Division:

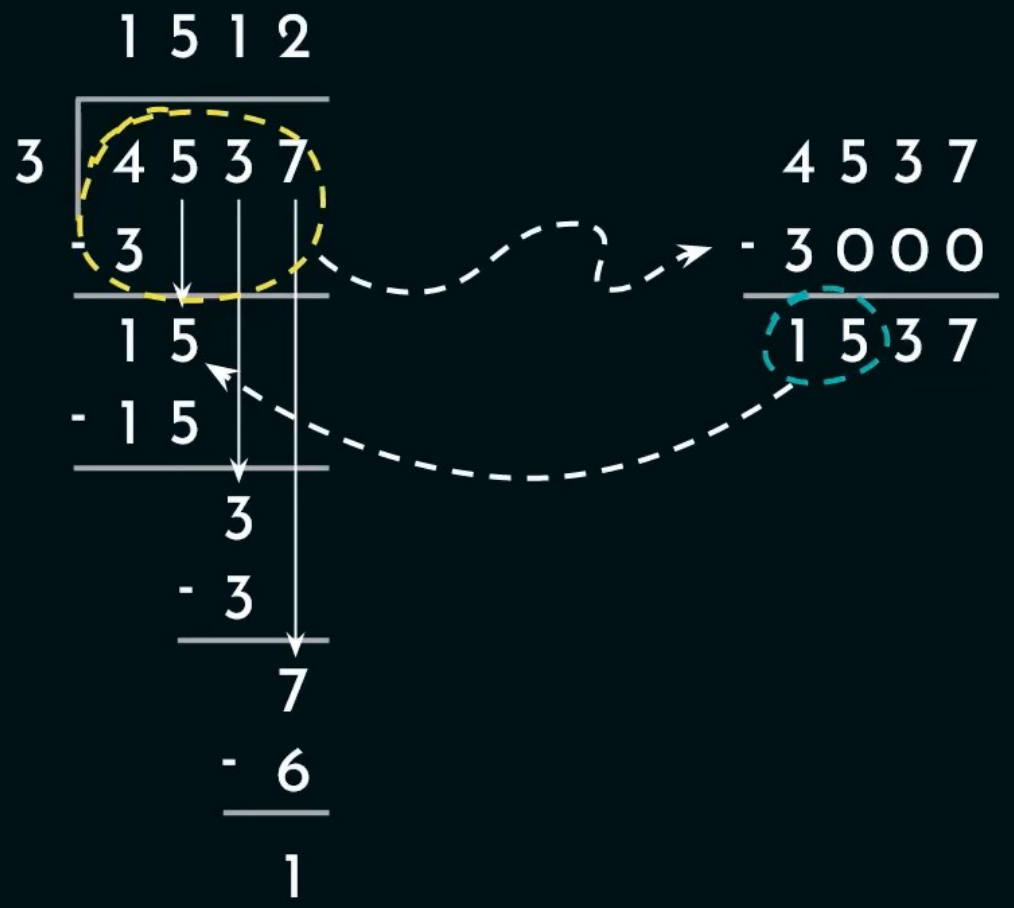
Dividend (d): 4 5 3 7 Divisor (v): 3





Restoring Division:

Dividend (d): 4 5 3 7 Divisor (v): 3



$q_3\ q_2\ q_1\ q_0$

q → 1 5 1 2

v → 3 | 4 5 3 7

- 3 0 0 0 $v \times q_3 \times 10^3$

1 5 3 7

- 1 5 0 0 $v \times q_2 \times 10^2$

0 0 3 7

- 0 0 3 0 $v \times q_1 \times 10^1$

0 0 0 7

- 0 0 0 6 $v \times q_0 \times 10^0$

0 0 0 1

Restoring Division:

Dividend (d): 4 5 3 7 Divisor (v): 3

$v \times q_3 \times 10^3$

4 5 3 7 - 3 x 1 x 10^3 = 1 5 3 7

$v \times q_2 \times 10^2$

1 5 3 7 - 3 x 5 x 10^2 = 0 0 3 7

$v \times q_1 \times 10^1$

0 0 3 7 - 3 x 1 x 10^1 = 0 0 0 7

$v \times q_0 \times 10^0$

0 0 0 7 - 3 x 2 x 10^0 = 0 0 0 1

$q_3\ q_2\ q_1\ q_0$

q ⟶ 1 5 1 2

v → 3 | 4 5 3 7

- 3 0 0 0 $v \times q_3 \times 10^3$

r(3) 1 5 3 7

- 1 5 0 0 $v \times q_2 \times 10^2$

r(2) 0 0 3 7

- 0 0 3 0 $v \times q_1 \times 10^1$

r(1) 0 0 0 7

- 0 0 0 6 $v \times q_0 \times 10^0$

r(0) 0 0 0 1

Restoring Division:

Dividend (d): 4 5 3 7 Divisor (v): 3

$$
\begin{array}{llll}
 & v \times q_3 \times 10^3 & & \\
4\,5\,3\,7 - & 3 \times 1 \times 10^3 & = & 1\,5\,3\,7 \\
 & v \times q_2 \times 10^2 & & \\
1\,5\,3\,7 - & 3 \times 5 \times 10^2 & = & 0\,0\,3\,7 \\
 & \vdots & &
\end{array}
$$

Generalization:

$r(i) = r(i+1) - q_i \times v \times 10^i$ where, $i = n-1, n-2, \ldots, 1, 0$

	q_3	q_2	q_1	q_0	
q →	1	5	1	2	
v → 3	4	5	3	7	
	- 3	0	0	0	$v \times q_3 \times 10^3$
r(3)	1	5	3	7	
	- 1	5	0	0	$v \times q_2 \times 10^2$
r(2)	0	0	3	7	
	- 0	0	3	0	$v \times q_1 \times 10^1$
r(1)	0	0	0	7	
	- 0	0	0	6	$v \times q_0 \times 10^0$
r(0)	0	0	0	1	

Restoring Division:

Dividend (d): 4 5 3 7 Divisor (v): 3

$v \times q_3 \times 10^3$

$4537 - 3 \times 1 \times 10^3 = 1537$

$v \times q_3 \times 10^3$

$4537 - 3 \times 2 \times 10^3 = -1463$

	q_3	q_2	q_1	q_0	
q →	1	5	1	2	
v → 3	4	5	3	7	
	- 3	0	0	0	$v \times q_3 \times 10^3$
r(3)	1	5	3	7	
	- 1	5	0	0	$v \times q_2 \times 10^2$
r(2)	0	0	3	7	
	- 0	0	3	0	$v \times q_1 \times 10^1$
r(1)	0	0	0	7	
	- 0	0	0	6	$v \times q_0 \times 10^0$
r(0)	0	0	0	1	

Restoring Division:

Dividend (d): 4 5 3 7 Divisor (v): 3 Quotient (q): q_3 q_2 q_1 q_0 = 1 5 1 2 Remainder (r): 1

$4537 - 3 \times 10^3 = 1537$ $q_3 = 1$

$1537 - 3 \times 10^3 = -1463$ $q_3 = 2$

$-1463 + 3 \times 10^3 = 1537$ $q_3 = 1$

$1537 - 3 \times 10^2 = 1237$ $q_2 = 1$

$1237 - 3 \times 10^2 = 937$ $q_2 = 2$

$937 - 3 \times 10^2 = 637$ $q_2 = 3$

$637 - 3 \times 10^2 = 337$ $q_2 = 4$

$337 - 3 \times 10^2 = 37$ $q_2 = 5$

$37 - 3 \times 10^2 = -263$ $q_2 = 6$

$-263 + 3 \times 10^2 = 37$ $q_2 = 5$

$37 - 3 \times 10^1 = 7$ $q_1 = 1$

$7 - 3 \times 10^1 = -23$ $q_1 = 2$

$-23 + 3 \times 10^1 = 7$ $q_1 = 1$

$7 - 3 \times 10^0 = 4$ $q_0 = 1$

$4 - 3 \times 10^0 = 1$ $q_0 = 2$

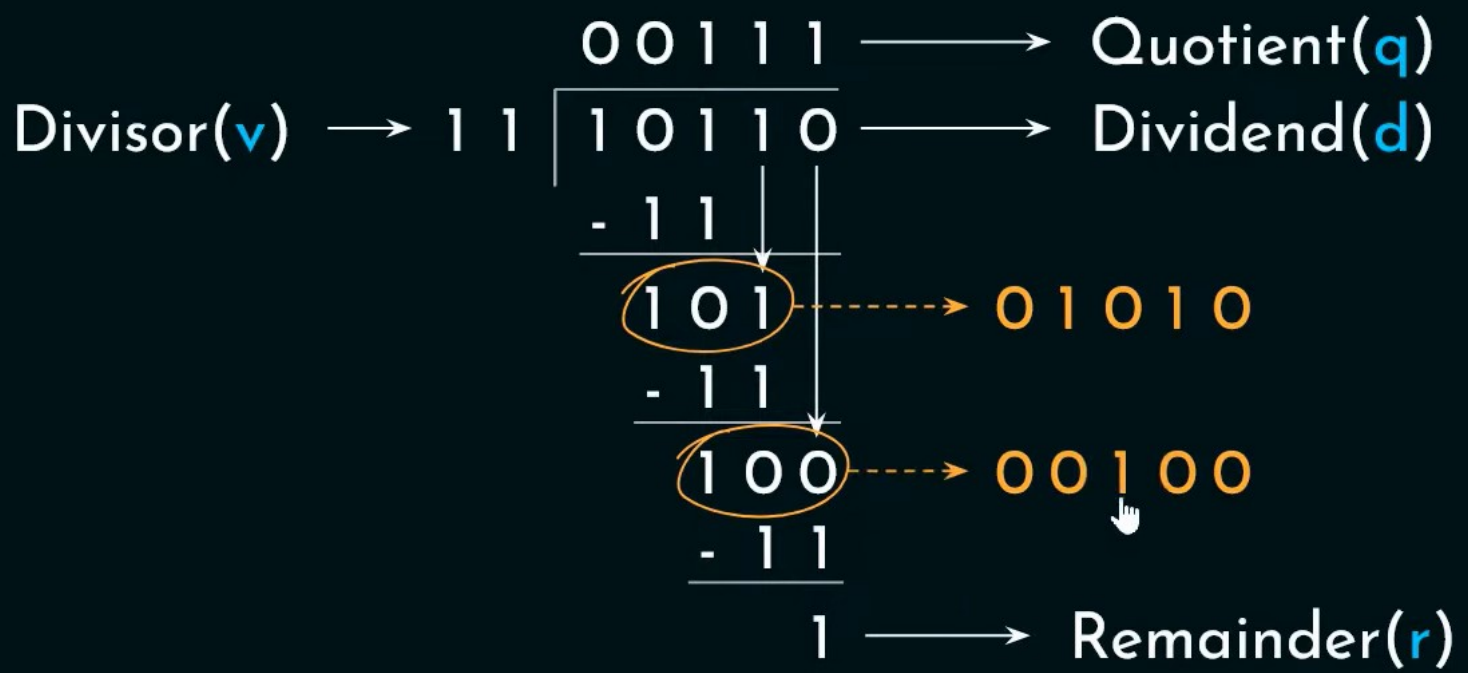
$1 - 3 \times 10^0 = -2$ $q_0 = 3$

$-2 + 3 \times 10^0 = 1$ $q_0 = 2$

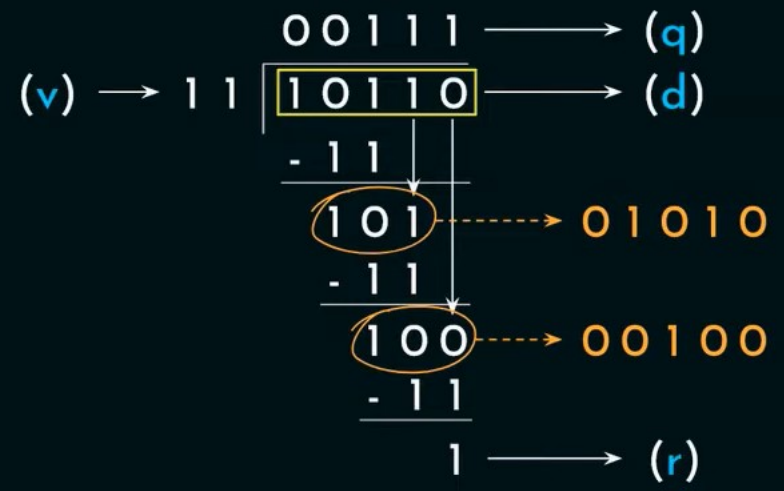
Restoring Division:

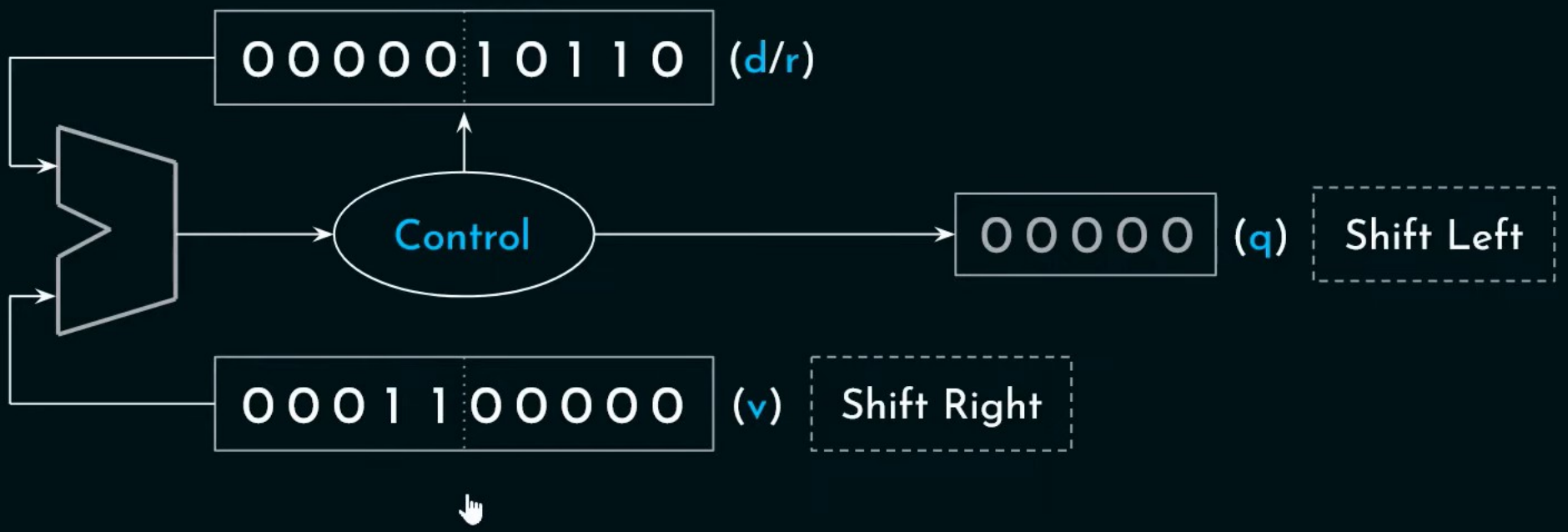
Dividend(d): $(1\ 0\ 1\ 1\ 0)_2 = (22)_{10}$

Divisor(v): $(1\ 1)_2 = (3)_{10}$

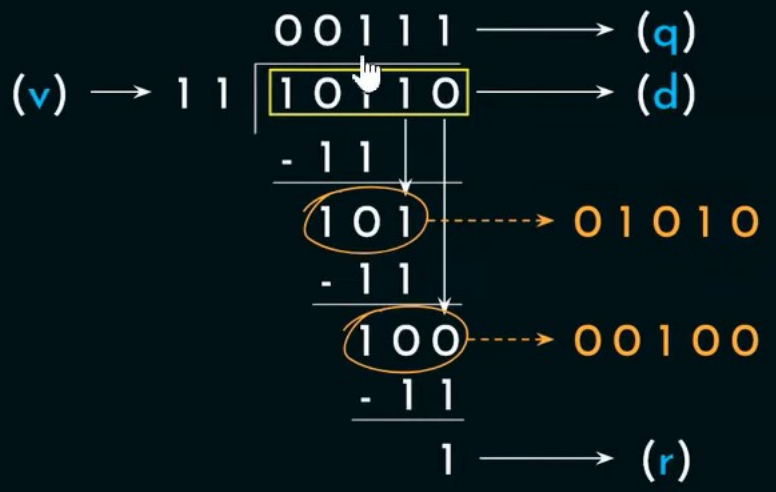


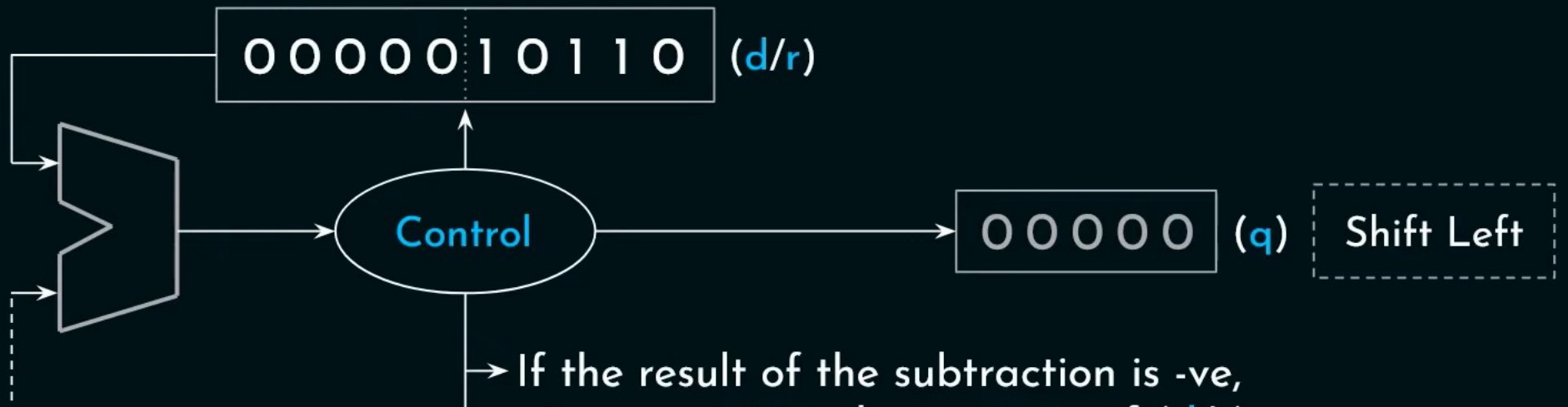
Restoring Division:





Restoring Division:





If the result of the subtraction is -ve,

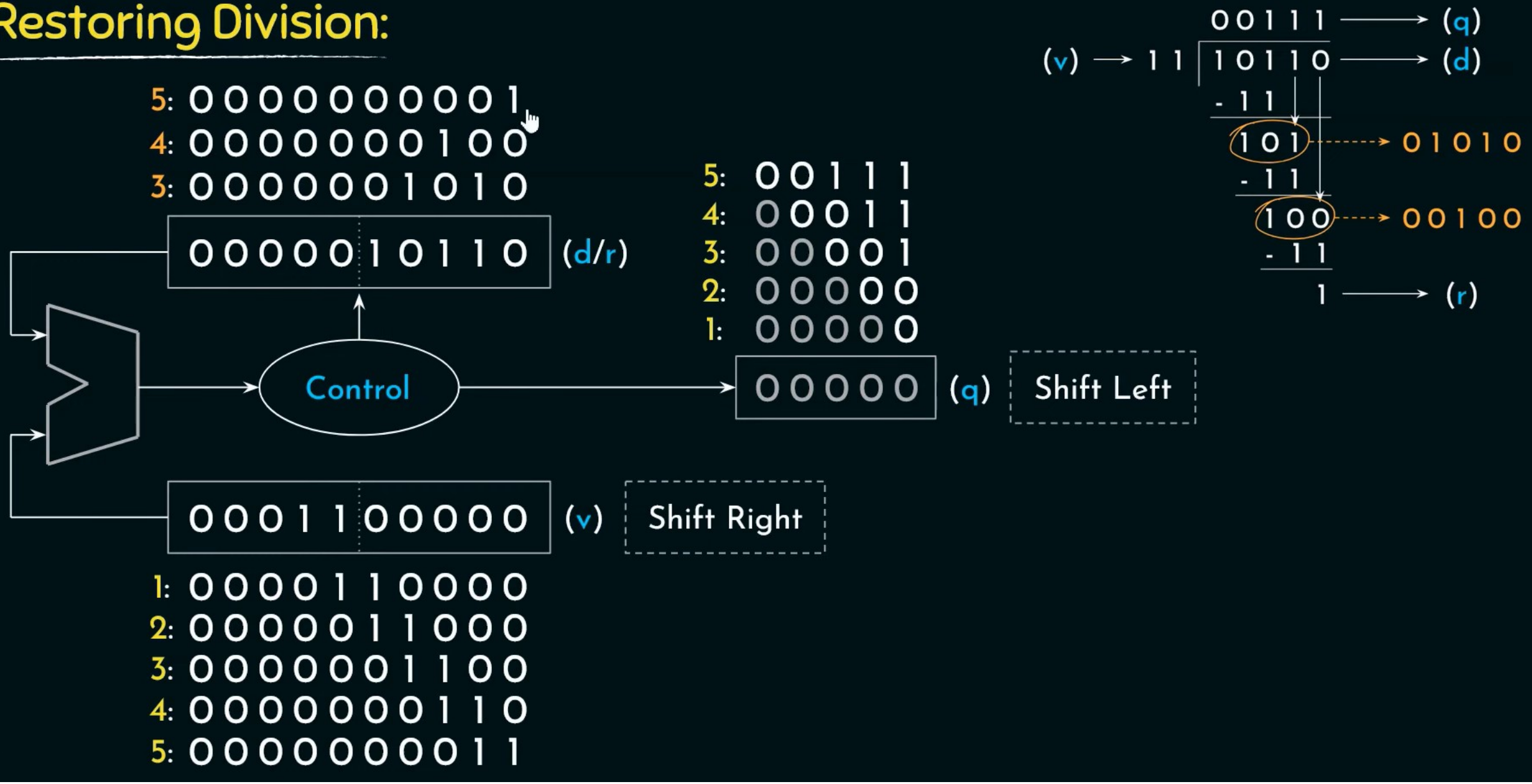
1. we restore the contents of (d/r).
2. Shift (q) and update the LSb with 0.

If the result of the subtraction is +ve,

1. we update the contents of (d/r).
2. Shift (q) and update the LSb with 1.

$$\begin{array}{r} 1 \\ -\ 1\ 1 \\ \hline \end{array} \Big/ \begin{array}{r} 1\ 0 \\ -\ 1\ 1 \\ \hline \end{array}$$

Restoring Division:

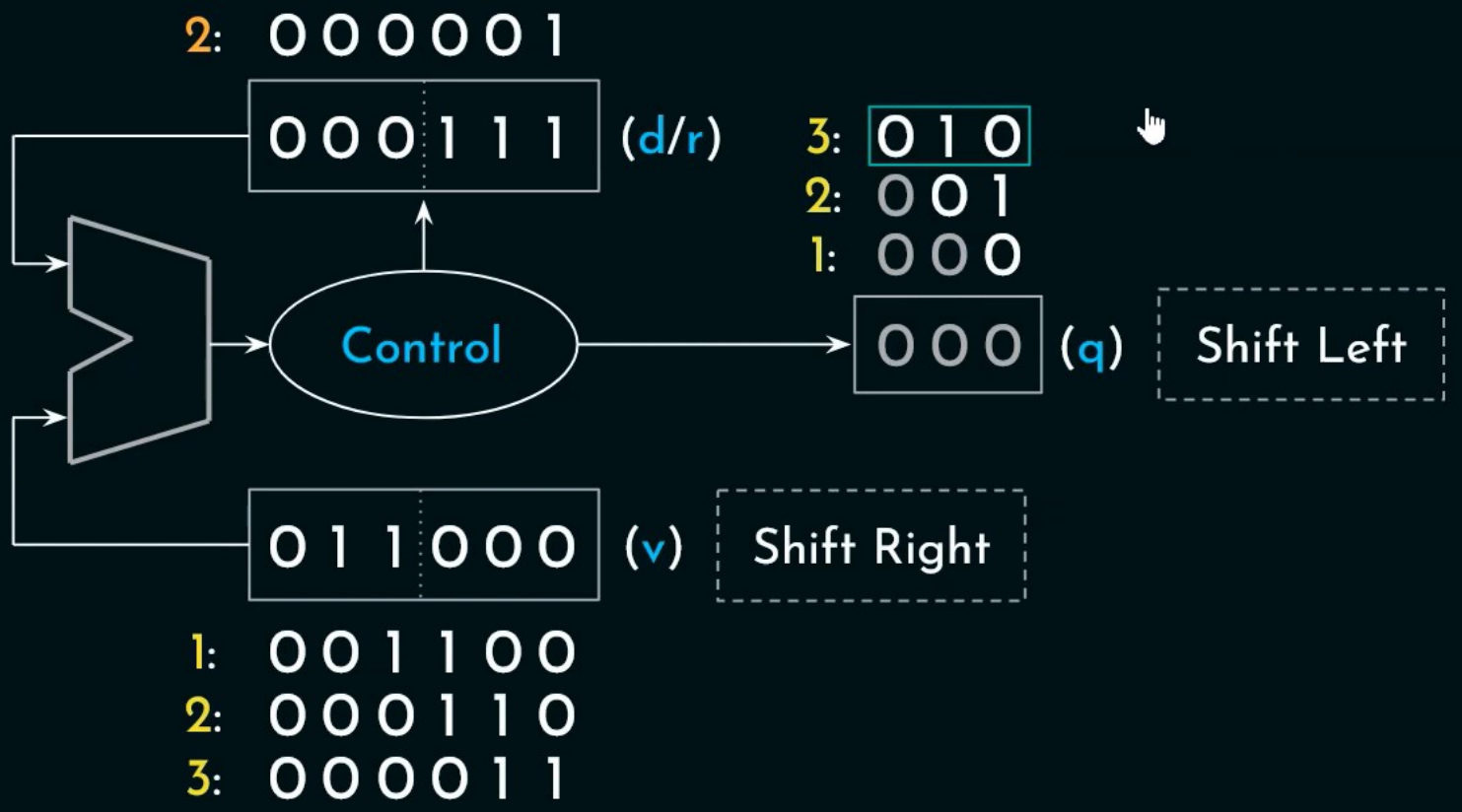


Restoring Division:

Dividend(d): $(111)_2 = (7)_{10}$

Divisor(v): $(11)_2 = (3)_{10}$

```
      0 1 0
    ---------
1 1 | 1 1 1
    - 1 1
    -------
        1
```

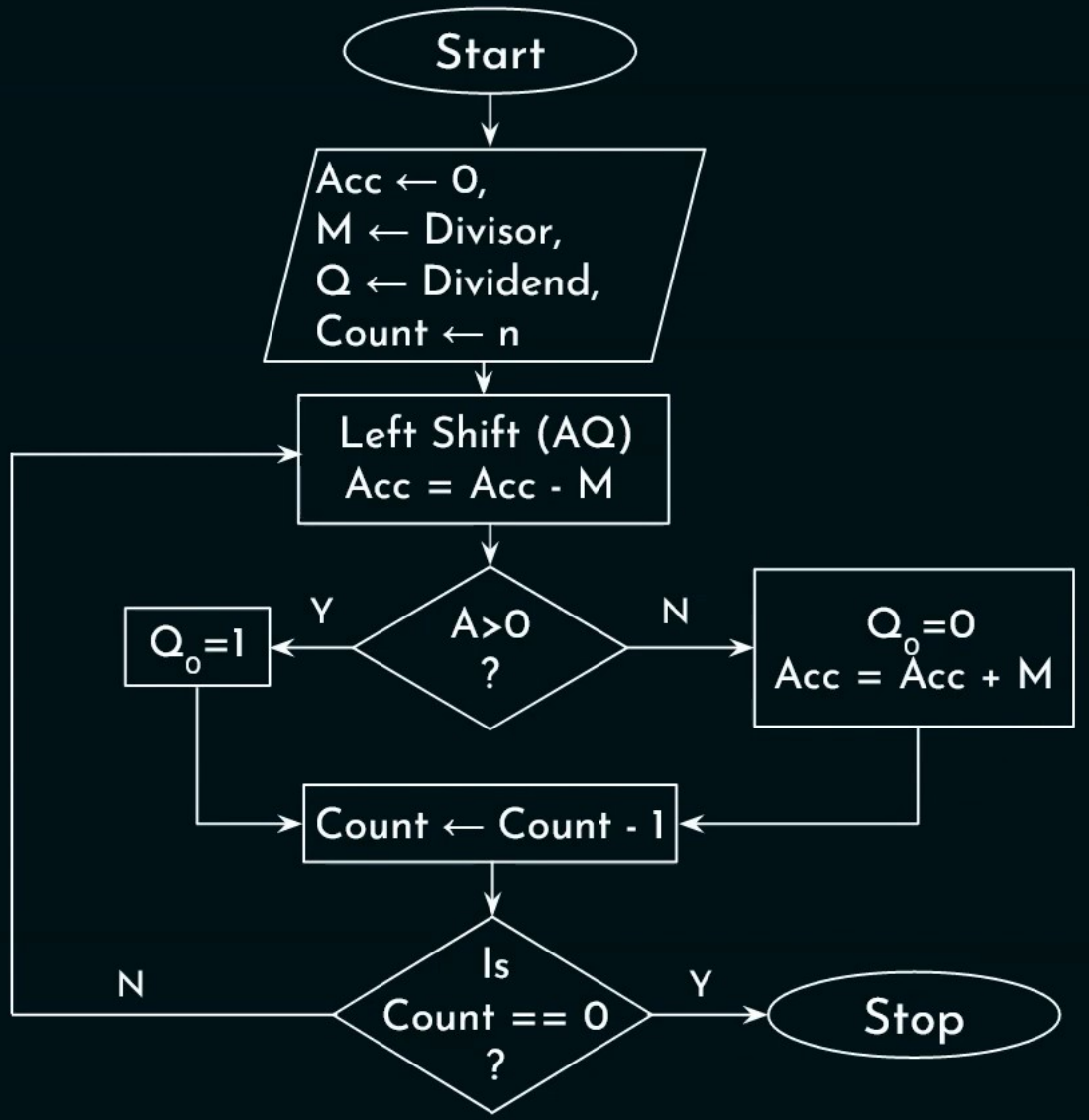


Restoring Division:

Dividend(d): $(111)_2 = (7)_{10}$

Divisor(v): $(11)_2 = (3)_{10}$

	M	A_{cc}	Q	Count
	0 1 1	0 0 0	1 1 1	3
1: LS(AQ)		0 0 1	1 1 _	
A=A-M -ve		0 0 1	1 1 0	2
2: LS(AQ)		0 1 1	1 0 _	
A=A-M +ve		0 0 0	1 0 1	1
3: LS(AQ)		0 0 1	0 1 _	
A=A-M -ve		0 0 1	0 1 0	0



Non-restoring Division:

1. If A_{cc}>0, then 1. Left Shift (AQ)
   2. $A_{cc} = A_{cc}$ - M

   else, 1. Left Shift (AQ)
   2. $A_{cc} = A_{cc}$ + M
2. If A_{cc}>0, then Q_0= 1, else, Q_0= 0

If A_{cc}<0, Restore A_{cc}

	M	A_{cc}	Q	Count
	0 1 1	0 0 0	1 1 1	3
1:	LS(AQ)	0 0 1	1 1 _	
		- 0 1 1		
	A=A-M	1 1 0	1 1 0	2
2:	LS(AQ)	1 0 1	1 0 _	
		+ 0 1 1		
	A=A+M	0 0 0	1 0 1	1
3:	LS(AQ)	0 0 1	0 1 _	
		- 0 1 1		
	A=A-M	1 1 0	0 1 0	0

Restore

0 0 1

Non-restoring Division:

1. If A_{cc}>0, then 1. Left Shift (AQ)
   2. $A_{cc} = A_{cc} - M$

   else, 1. Left Shift (AQ)
   2. $A_{cc} = A_{cc} + M$
2. If A_{cc}>0, then Q_0= 1, else, Q_0= 0

If A_{cc}<0, Restore A_{cc}

	M	A_{cc}	Q	Count
	0 1 1	0 0 0	1 1 1	3
1: LS(AQ)		0 0 1	1 1 _	
		- 0 1 1		
A=A-M		1 1 0	1 1 0	2
2: LS(AQ)		1 0 1	1 0 _	
		+ 0 1 1		
A=A+M		0 0 0	1 0 1	1
3: LS(AQ)		0 0 1	0 1 _	
		- 0 1 1		
A=A-M		1 1 0	0 1 0	0
		↓ Restore		
		0 0 1